



UPPSALA
UNIVERSITET

UPTEC F19 003

Examensarbete 30 hp
Februari 2019

Design of a UAV-based radio receiver for avalanche beacon detection using software defined radio and signal processing

Richard Hedlund



UPPSALA
UNIVERSITET

**Teknisk- naturvetenskaplig fakultet
UTH-enheten**

Besöksadress:
Ångströmlaboratoriet
Lägerhyddsvägen 1
Hus 4, Plan 0

Postadress:
Box 536
751 21 Uppsala

Telefon:
018 – 471 30 03

Telefax:
018 – 471 30 00

Hemsida:
<http://www.teknat.uu.se/student>

Abstract

Design of a UAV-based radio receiver for avalanche beacon detection using software defined radio and signal processing

Richard Hedlund

A fully functional proof of concept radio receiver for detecting avalanche beacons at the frequency 457 kHz was constructed in the work of this master thesis. The radio receiver is intended to be mounted on an unmanned aerial vehicle (UAV or drone) and used to aid the mountain rescue teams by reducing the rescue time in finding avalanche victims carrying a transmitting beacon. The main parts of this master thesis involved hardware requirement analysis, software development, digital signal processing and wireless communications.

The radio receiver was customized to receive low power signal levels because magnetic antennas are used and the avalanche beacon will operate in the reactive near field of the radio receiver. Noise from external sources has a significant impact on the performance of the radio receiver.

This master thesis allows for straightforward further development and refining of the radio receiver due to the flexibility of the used open-source software development kit GNU Radio where the digital signal processing was performed.

Handledare: Johan Tenstam
Ämnesgranskare: Mikael Sternad
Examinator: Tomas Nyberg
ISSN: 1401-5757, UPTec F19 003

Populärvetenskaplig sammanfattning

I detta examensarbete har en fullt funktionell "proof of concept" radiomottagare för detektering av lavintransceivers på frekvensen 457 kHz konstruerats. Den framtagna radiomottagaren ska monteras på en drönare för att kunna underlätta för fjällräddningen genom att förminska söktiden av begravda lavinoffer som har en aktiverad lavintransceiver på sig. Huvuddelarna i examensarbetet involverade kravanalys av hårdvara, mjukvaruutveckling, digital signalbehandling och trådlös kommunikation.

Radiomottagaren var framtagen för att kunna ta emot låg signalstyrka, eftersom magnetiska antenner används. Lavintranscievern kommer befinna sig i det reaktiva närfället av radiomottagaren. Brus och störningar från externa källor har en markant effekt på prestandan hos radiomottagaren.

Detta examensarbete förenklar framtida utveckling och förbättring av radiomottagaren tack vare flexibiliteten hos verktyget GNU Radio, som har använts för den digitala signalbehandlingen.

Contents

1	Introduction	1
1.1	Preface	1
1.2	Project description	1
1.3	Project objectives	2
1.4	System overview	3
2	Theory	4
2.1	ETSI EN 300 718-1	4
2.2	Software-defined radio	4
2.3	Digital signal processing	5
2.3.1	Sampling theorem	5
2.3.2	Quantization	7
2.3.3	Dynamic range	8
2.3.4	Discrete-time systems and digital filters	8
2.3.4.1	FIR-filters	9
2.3.5	Windowing functions	10
2.3.6	Matched filter	11
2.3.7	Discrete fourier transform	12
2.3.8	Decimation	12
2.3.9	Digital down conversion	12
2.3.10	Modulation	13
2.3.10.1	On-off keying	13
2.3.10.2	IQ data and modulation	14
2.4	Noise	14
2.5	Serial communication	14
2.5.1	UART	15
2.5.2	Mavlink protocol	15
2.6	Ferrite rod antenna	16
2.7	Link budget	17
3	Hardware	18
3.1	Radio receiver hardware	18
3.1.1	SDR: AirSpy HF+	18
3.1.2	Single-board computer: Odroid XU4	20
3.1.3	Ferrite rod antennas and LNA	21
3.2	Test equipment	22
3.2.1	Signal generator	22
3.2.2	Logic analyzer	22
3.2.3	Avalanche beacon	23
4	Software	24
4.1	Linux	24
4.2	GNU Radio	24
4.3	ZeroMQ	25
5	Implementation	26

5.1	DSP-chain	26
5.1.1	FFT	26
5.1.2	Filter bank	28
5.1.3	Matched filter	30
5.2	Software written in Python	31
5.3	System tests	32
5.3.1	Lab bench test	33
5.3.2	Field test	33
6	Results	34
6.1	Matched filter	35
6.1.1	Lab bench Test	35
6.1.2	Field test	36
6.2	Filter bank	40
6.2.1	Lab bench test	40
6.2.2	Field test	40
6.3	FFT	41
6.3.1	Lab bench test	41
6.3.2	Field test	41
6.4	EMI measurements	42
6.5	UART communication and Mavlink	42
7	Discussion	43
8	Conclusions	44

Acronyms

ADC	Analog-to-Digital Conversion
AGC	Automatic Gain Control
ASK	Amplitude-Shift Keying
CT	Continuous-Time
CPU	Central Processing Unit
DDC	Digital Down Conversion
DFT	Discrete Fourier Transform
DSP	Digital Signal Processing
DT	Discrete Time
dB	Decibels
dBm	Power expressed in decibels with reference to one milliwatt [mW]
EMI	Electromagnetic Interference
FFT	Fast Fourier Transform
FIR	Finite Impulse Response
FM	Frequency Modulation
FPGA	Field Programmable Gate Array
GPIO	General Purpose Input/Output
GRC	Gnuradio Companion
GUI	Graphical User Interface
IC	Integrated Circuit
IDE	Integrated Development Environment
IIR	Infinite Impulse Response
I2C	Inter-Integrated Circuit
I/O	Input / Output
LNA	Low Noise Amplifier
LTi	Linear Time-Invariant
MDS	Minimum Detectable Signal
OOK	On-Off Keying
RAM	Random Access Memory
RF	Radio Frequency

RMS	Root Mean Square
SDR	Software-Defined Radio
SNR	Signal-to-Noise Ratio
SNQR	Signal-to-Quantization-Noise Ratio
SoC	System on a Chip
UART	Universal Asynchronous Receiver Transmitter
UAV	Unmanned Aerial Vehicle
USB	Universal Serial Bus
UX	User Experience

1 Introduction

1.1 Preface

This master thesis is a part of a larger project at ÅF Digital Solutions AB in Uppsala and Solna, which together with Integrated UAV, the Swedish Police and the Swedish Mountain Rescue, runs the larger project financed by Vinnova. The main goal of the larger project is to show how modern radio technology and signal processing combined with the UAVs ability to systematically and quickly search a big area to locate a buried avalanche transceiver could be utilized to aid the mountain rescue team and shorten the rescue time. The larger project consists of three major parts which are the following:

- UAV and flight controller system (called the UAV-system in this report)
- Radio Receiver (focus of this master thesis)
- User Experience (UX)

1.2 Project description

The primary goal of this master thesis is to design a UAV-based radio receiver that should be able to detect an avalanche beacon carried by a victim buried in snow. The finished radio receiver should detect the signal strength and the direction of a transmitting avalanche beacon at the frequency 457kHz.

The radio receiver should be constructed with two antennas, a low-noise amplifier (LNA), two software defined radios (SDRs), digital signal processing (DSP) and a single-board computer. The two SDRs are going to be connected to the single-board computer that continuously detects the signature of the avalanche beacon and sends measured signal strength and direction to the UAV-system. To achieve this a peripheral connection using a convenient interface and protocol is needed. The DSP-chain is to be developed in GNU Radio, which is explained in greater detail in section 4.2, and should run in real-time. The antennas and LNA should be constructed to minimize thermal noise. It is expected that a receiver at these low frequencies will be affected by electromagnetic interference (EMI) from the UAV itself, which should be considered during the development of the design.

The primary focus of this master thesis is the development of the DSP, external software for the complete detection algorithm and interaction with the hardware between the single-board computer and the UAV-system. The two most important goals are the following:

The analog front-end with the antennas and LNA is provided from experts within the project, and is beyond the scope of this master thesis.

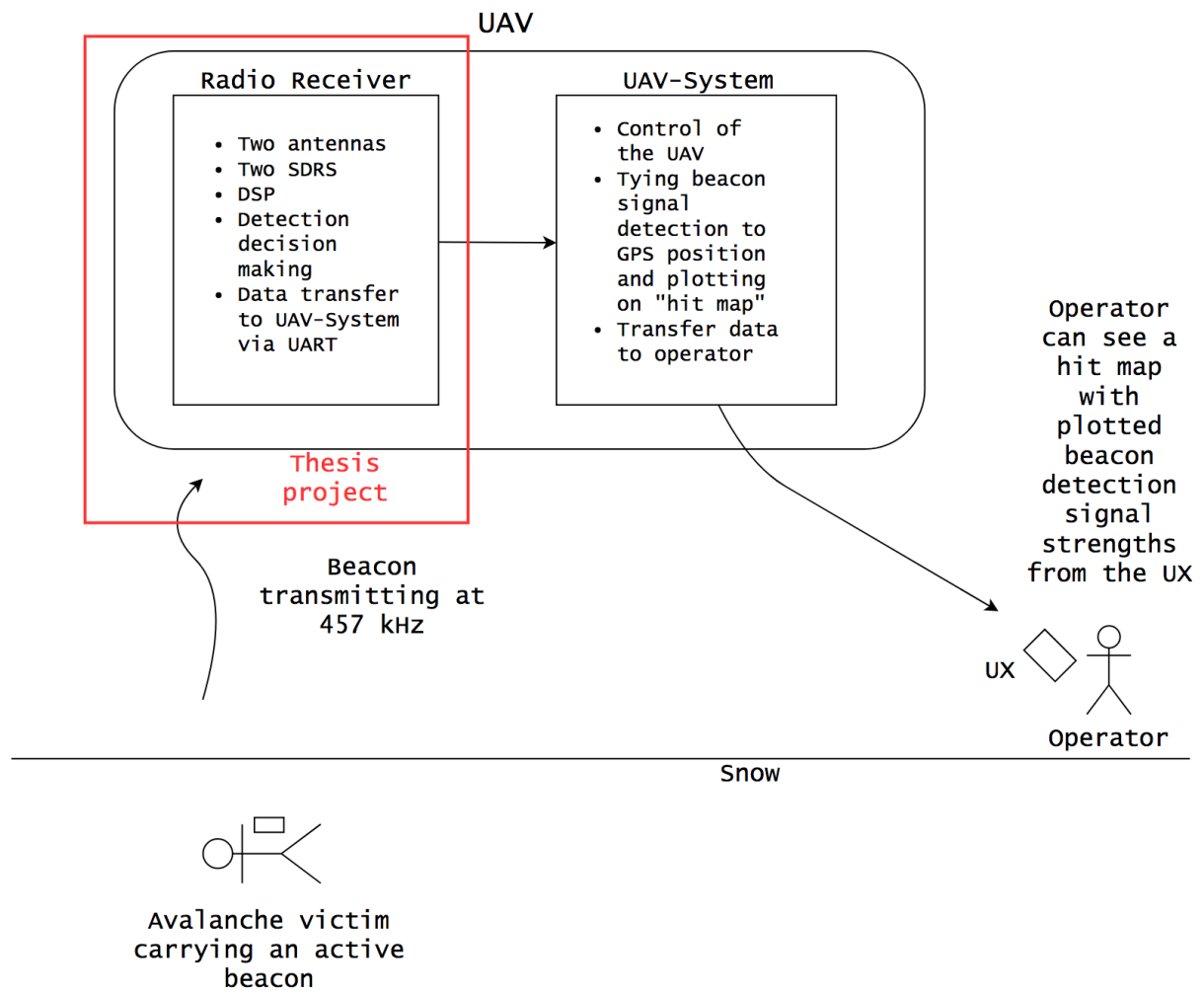


Figure 1.1: Overall overview of the larger project.
The focus of this master thesis is highlighted in red.

1.3 Project objectives

The finished radio receiver system should be able to detect the signature, signal strength and direction of an avalanche beacon, and then report the time of measurement, signal strength and direction to the UAV-system. The performance of the finished radio receiver depends heavily on the dynamic range and should be designed to maximize the potential of the SDRs to utilize their full dynamic range. An overall overview of the system to be implemented can be seen in figure 1.1.

1.4 System overview

A block diagram representation of the detailed radio receiver can be seen in figure 1.2. Two SDRs (AirSpy HF+) and two antennas that are perpendicular to each other so that one can determine a direction towards the transmitting avalanche beacon by comparing the two signal strengths. The DSP and Python implementations runs in real-time on the single-board computer which is vital for the project. The full implementation is explained in section 5.

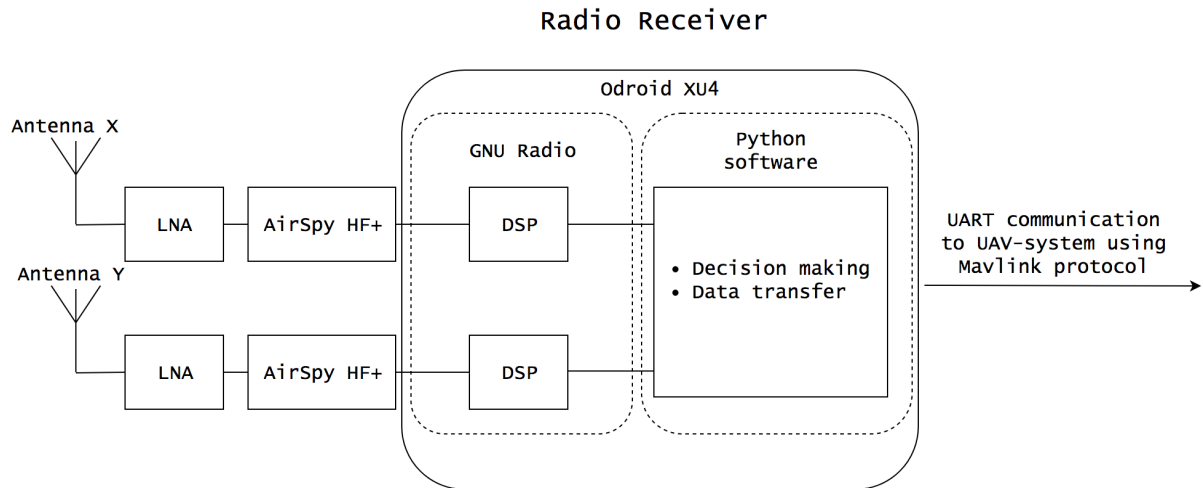


Figure 1.2: Detailed overview of the implemented radio receiver.

2 Theory

2.1 ETSI EN 300 718-1

ETSI EN 300 718-1, is a European standard governing the technical aspects of avalanche beacon transceivers [14]. From ETSI EN 300 718-1 the following information about an avalanche beacon signal can be extracted:

- Frequency of operation: 457 kHz.
- Frequency limits: 457 kHz $\pm$ 80 Hz.
- Modulation is of the type On-Off Keying (OOK).
- On-time: 70 ms minimum.
- Off-time: 400 ms minimum.
- Period: 1000 ms $\pm$ 300 ms (on-time plus off-time).

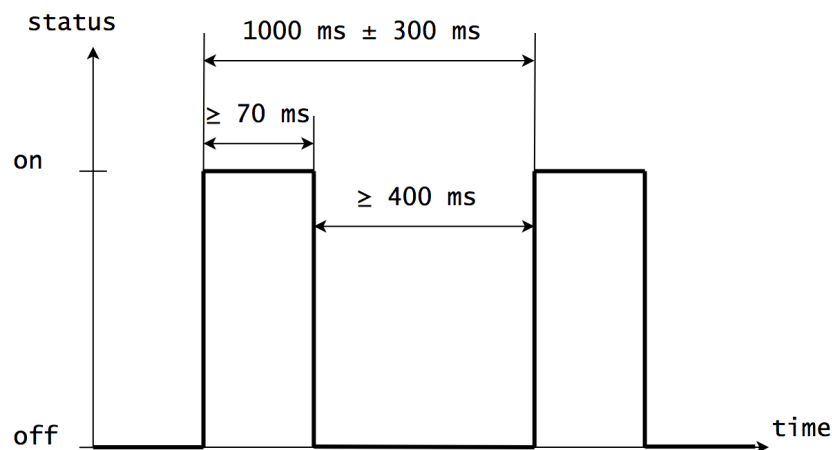


Figure 2.1: Carrier keying of an avalanche beacon governed by ETSI EN 300 718-1.

An illustration of the avalanche beacon signal properties can be seen in figure 2.1.

2.2 Software-defined radio

Software-defined radio (SDR) is a communication system where the transmitter and receiver concepts to a large part are realized in software instead of dedicated integrated circuits in hardware. The signal processing is carried out in a host PC, dedicated DSP, FPGA or an embedded system [4].

One major advantage of SDR systems is flexibility due to the ability to adjust the signal processing to match certain requirements, thus making it easier to keep products up-to-date longer periods of time. SDRs are also relatively cheap compared to the cost of constructing an equivalent analog radio communication system using hardware.

The general architecture an SDR receiver consists of the following parts: analog RF front-end, digital front-end and digital baseband signal processor.

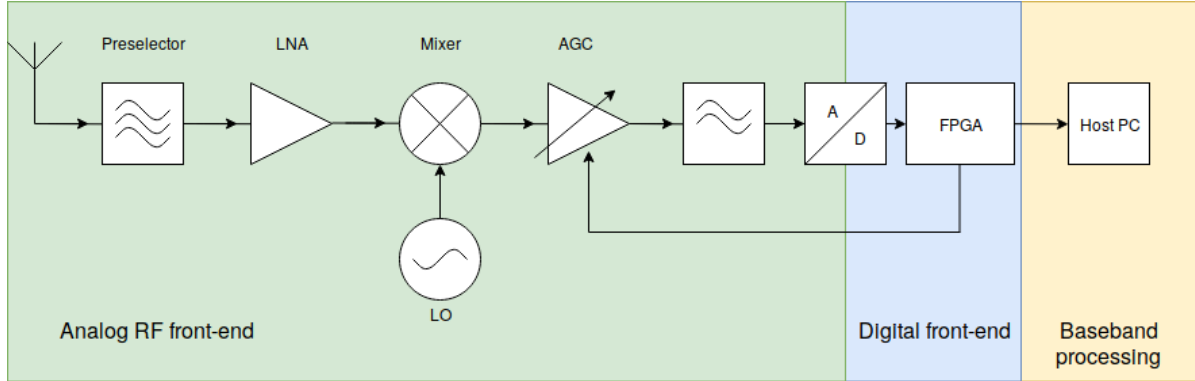


Figure 2.2: SDR receiver architecture

Figure 2.2 shows the different parts of the SDR receiver architecture.

2.3 Digital signal processing

Digital signal processing (DSP) have three main steps [8] for manipulating analog signals which are the following:

1. Analog to digital (A/D) conversion is the process of digitizing an analog signal, where the analog signal is sampled and each sample is quantized to a finite number of bits.
2. The digital samples are processed by a digital signal processor.
3. The digital samples can be converted back to analog form using digital to analog (D/A) conversion, also called reconstruction.

In this project the SDR takes care of the first step with digitizing the signal, seen in figure 3.1. GNU Radio is used in the second step to perform the digital signal processing chain which is explained in greater detail in section 5.1.

2.3.1 Sampling theorem

The sampling process [8] of an analog signal $x(t)$ is illustrated in figure 2.3, where the analog signal is measured in equally spaced intervals every T seconds according to equation (2.1).

$$t = nT. \quad n = 0, 1, 2, \dots \quad (2.1)$$

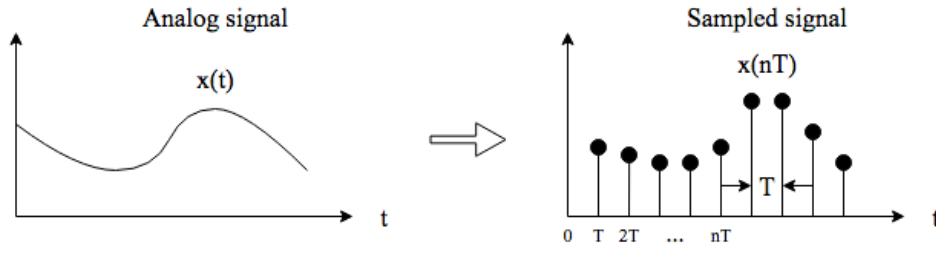


Figure 2.3: An analog signal $x(t)$ sampled according to equation (2.1), resulting in the sampled signal $x(nT)$ [8].

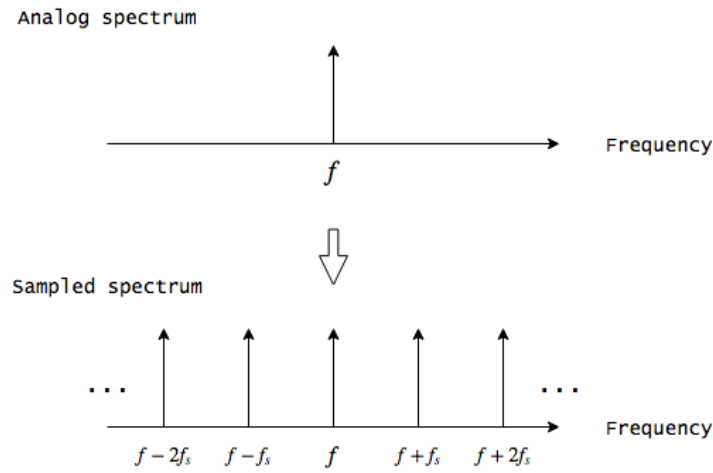


Figure 2.4: The resulting frequency spectrum of the sampled signal $x(nT)$ [8].

The process of sampling an analog signal will introduce high-frequency components into the spectrum of the sampled signal which can be observed in figure 2.4. For this particular reason it is important to have an idea of what effect the sampling has on the frequency spectrum. Looking at figure 2.4, one cannot distinguish the unique original frequency of the analog signal because it could be any of the replicated frequencies. This effect is called *aliasing* and can be avoided if the sampling theorem is satisfied [8]. There are two criteria provided by the sampling theorem that needs to be satisfied to correctly represent an analog signal $x(t)$ by the sampled signal $x(nT)$, the two conditions are:

1. The analog signal $x(t)$ must be band limited, meaning its frequency spectrum must be limited and only contain frequencies up to some maximum frequency $f_{\max}$. An illustration of a band limited frequency spectrum can be seen in figure 2.5.
2. The sampling rate f_s must be at least twice the maximum frequency $f_{\max}$, according to equation (2.2).

$$f_s \geq 2f_{\max}. \quad (2.2)$$

The sampling frequency is related to the sampling interval according to equation (2.3).

$$f_s = 1/T. \quad (2.3)$$

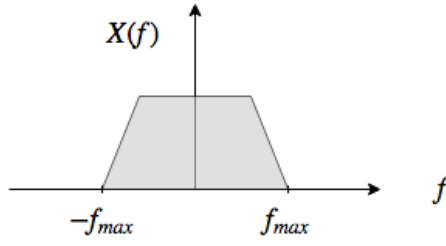


Figure 2.5: Band limited frequency spectrum $X(f)$ [8].

According to equation (2.2), the minimum allowed sampling rate is $f_s = 2f_{\max}$. This minimum allowed sampling rate is called the *Nyquist rate* and $f_s/2$ is called the *Nyquist frequency* [8]. The *Nyquist frequency* interval is defined according to equation (2.4).

$$\left[-\frac{f_s}{2}, \frac{f_s}{2} \right]. \quad (2.4)$$

2.3.2 Quantization

The process of analog to digital conversion (A/D) includes a *sampler* and a *quantizer* [8]. The quantization step is where the analog sample is translated to a finite B-bit word represented by B-bits. After this step the quantized sample $x_Q(nT)$ can be processed digitally in the DSP. Figure 2.6 illustrates the quantization process.

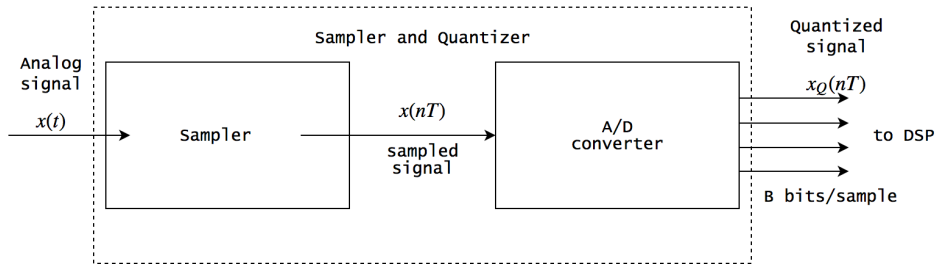


Figure 2.6: A/D conversion using a *sampler* and *quantizer* [8].

The quantized B-bit sample $x_Q(nT)$, takes only one of 2^B possible values. If the analog signal has a range R , the A/D converter divides this range into equally divided 2^B quantization levels. The difference between the quantization levels is called the *quantizer resolution* [8] and is given by equation (2.5).

$$Q = \frac{R}{2^B} \quad (2.5)$$

An illustration of a 3 bit, 2^3 level quantization is shown in figure 2.7

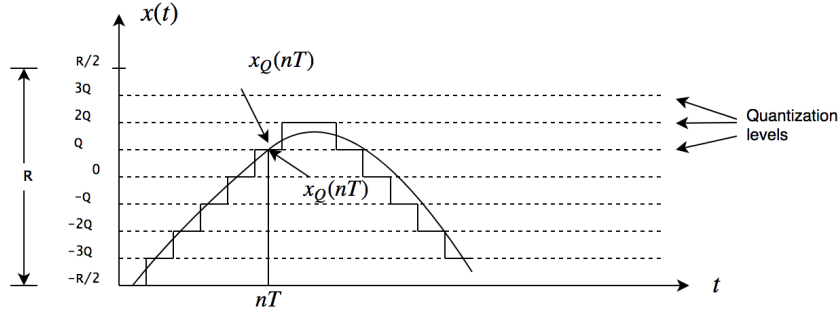


Figure 2.7: Quantization of a sampled signal $x(nT)$ [8].

The signal-to-quantization-noise ratio (SNQR) is the relationship between the range R and the quantization noise. Denoting the quantization noise as Q , the SNQR can be expressed as follows:

$$\text{SNQR} = 20 \log_{10} \left(\frac{R}{Q} \right) = B \cdot 20 \log_{10}(2) = 6B \text{ dB} \quad (2.6)$$

Equation (2.6) is referred to as the *6 dB per bit rule* and is the dynamic range of the quantizer [8]. This relationship can be used to determine the quantization requirements needed for a certain dynamic range.

2.3.3 Dynamic range

Dynamic range a term often used to describe the ratio between the strongest and weakest signal a receiver could handle with acceptable distortion or noise [4] in decibels. It is the ratio between a strong signal that causes a maximum tolerable distortion and weak signal that has the minimum tolerable signal-to-noise ratio (SNR).

Dynamic range can also have another definition if the receiver has a noisy surrounding with strong out-of-band signals degrading the signal performance of the receiver substantially [4].

2.3.4 Discrete-time systems and digital filters

Discrete-time systems, in particular linear time-invariant (LTI) systems, can be divided into infinite impulse response (IIR) or finite impulse response (FIR) types, depending on whether their respective impulse response is infinite or finite in time [1]. A discrete-time system is considered linear if the output signal, due to a linear combination of two or more input signals, can be obtained by forming the same linear combination of the individual outputs. A discrete-time system is considered time-invariant if a system remains unchanged over time, meaning that an input applied to the system today, should produce the same output if the same input was applied tomorrow [8].

The digital filter transforms a sequence of signal samples by changing its characteristics in the frequency domain [1]. A digital filter is characterized by the difference equation (2.7).

$$y[n] = b_0x[n] + b_1x[n-1] + \dots + b_Mx[n-M] - \dots - a_1y[n-1] - \dots - a_Ny[n-N], \quad (2.7)$$

Where the $a_1, a_2, \dots, a_N$ and $b_0, b_1, \dots, b_M$ are the filter coefficients, $y[n]$ and $x[n]$ are the output and input of the filter respectively. Equation (2.7) is an IIR-filter equation because of its recursive part containing a subtraction of previous outputs of the filter.

2.3.4.1 FIR-filters

The impulse response $H(z)$ of a FIR-filter approaches zero in time if the length N of the impulse response is finite. The FIR-filter only consists of the non-recursive part in equation (2.7), where the output y only depends on past inputs x . Hence the coefficients $b_0, b_1, \dots, b_M$ corresponds to an FIR-filter [1]. Taking the Z-transform of equation (2.7) and neglecting the recursive IIR-part we obtain the impulse response of the FIR-filter according to equation (2.8)

$$H(z) = \sum_{i=0}^{N-1} b_i z^{-i}, \quad (2.8)$$

where the filter coefficients $b_0, b_1, \dots, b_M$ are determined by the desired filter characteristics. Rewriting equation (2.8) into the discrete time (DT) domain gives equation (2.9).

$$H[z] = \sum_{k=0}^{N-1} h[k] z^{-k} = h[0] + h[1]z^{-1} + h[2]z^{-2} + \dots + h[N-1]z^{-(N-1)} \quad (2.9)$$

The window method can be used in FIR-filter design, where a window function $w[k]$ is used to truncate the impulse response of an ideal IIR low- or highpass filter to a finite length N . The impulse responses of the ideal low pass, high pass and band pass filters are:

$$\text{Low pass:} \quad h_{ilp}[k] = \begin{cases} \frac{\sin[(k-m)\Omega_c]}{(k-m)\pi} & \text{if } k \neq N/2 \\ \frac{\Omega_c}{\pi} & \text{if } k = N/2. \end{cases} \quad (2.10)$$

$$\text{High pass:} \quad h_{ihp}[k] = \begin{cases} -\frac{\sin[(k-m)\Omega_c]}{(k-m)\pi} & \text{if } k \neq N/2 \\ 1 - \frac{\Omega_c}{\pi} & \text{if } k = N/2. \end{cases} \quad (2.11)$$

$$\text{Band pass:} \quad h_{ibp}[k] = \begin{cases} \frac{\sin[(k-m)\Omega_H]}{(k-m)\pi} - \frac{\sin[(k-m)\Omega_L]}{(k-m)\pi} & \text{if } k \neq N/2 \\ \frac{\Omega_H - \Omega_L}{\pi} & \text{if } k = N/2. \end{cases} \quad (2.12)$$

where $m = (N-1)/2$ and the normalized cutoff frequency Ω_c is defined as:

$$\Omega_c = 2\pi f_c T. \quad (2.13)$$

T is the sampling interval and f_c the desired cut-off frequency of the filter. Ω_L and Ω_H are the low and high cut-off frequencies of the band pass filter respectively.

By truncating the ideal low pass, high pass and band pass filters impulse responses with a windowing function $w[k]$, one can generate the three N -tap FIR-filters, with their filter coefficients calculated according to

$$\begin{aligned} h_{lp}[k] &= w[k]h_{ilp}[k], \\ h_{hp}[k] &= w[k]h_{ihp}[k], \\ h_{bp}[k] &= w[k]h_{ibp}[k]. \end{aligned} \quad (2.14)$$

2.3.5 Windowing functions

Commonly used window functions for digital filter design are the Bartlett (triangular), Hamming, Hanning and Blackman windows. They all have different properties in the frequency spectrum in terms of main lobe width and peak side lobe amplitude [1] (spectral leakage).

The four different window functions mentioned above have the following impulse responses:

$$w_{\text{bart}}[k] = \begin{cases} \frac{2k}{N-1} & \text{if } 0 \leq k \leq (N-1)/2, \\ 2 - \frac{2k}{N-1} & \text{if } (N-1)/2 < k \leq N-1, \\ 0 & \text{elsewhere.} \end{cases} \quad (2.15)$$

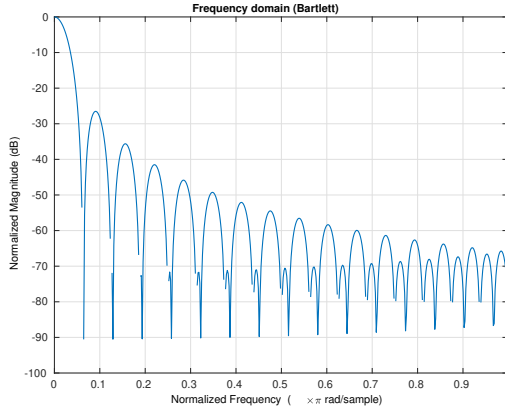
$$w_{\text{hamm}}[k] = \begin{cases} \alpha - \beta \cos\left(\frac{2\pi k}{N-1}\right) & \text{if } 0 \leq k \leq N-1 \\ 0 & \text{elsewhere.} \end{cases} \quad (2.16)$$

Where $\alpha = 0.54$ and $\beta = 1 - \alpha = 0.46$ [1].

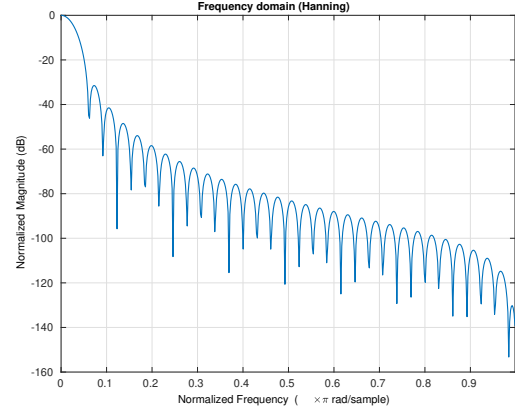
$$w_{\text{hann}}[k] = \begin{cases} 0.5 - 0.5 \cos\left(\frac{2\pi k}{N-1}\right) & \text{if } 0 \leq k \leq N-1 \\ 0 & \text{elsewhere.} \end{cases} \quad (2.17)$$

$$w_{\text{black}}[k] = \begin{cases} 0.42 - 0.5 \cos\left(\frac{2\pi k}{N-1}\right) + 0.08 \cos\left(\frac{4\pi k}{N-1}\right) & \text{if } 0 \leq k \leq N-1 \\ 0 & \text{elsewhere.} \end{cases} \quad (2.18)$$

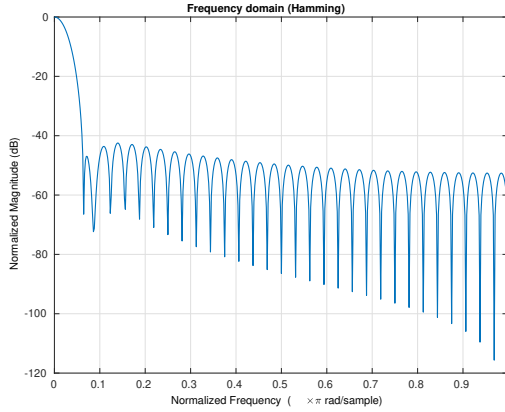
The frequency domain characteristics of windowing functions above can be seen in figure 2.8.



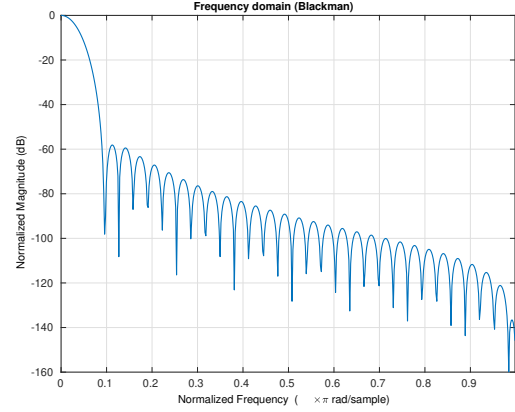
(a) Bartlett window.



(b) Hanning window.



(c) Hamming window.



(d) Blackman window.

Figure 2.8: Discrete time fourier transforms (DTFs) of the four window functions mentioned in section 2.3.5. The window length is $N = 64$. Plots were created in MATLAB.

2.3.6 Matched filter

A matched filter is obtained by correlating an unknown signal with a copy or template of a signal that one wants to detect in the unknown signal. This is called cross-correlation and is performed by convolving the unknown signal with the template. The matched filter is said to be optimal because it maximizes SNR in the receiver [3]. The convolution can be performed using an FIR filter of order M with impulse response $h[n]$, $n = 0, 1, 2, \dots, M$. Here, the impulse response $h[n]$ is the template to cross-correlate with the input signal [8]. In mathematical terms this is described in equation (2.19).

$$y[n] = \sum_m h[m] \cdot x[n - m], \quad 0 \leq m \leq M, \quad (2.19)$$

where $y[n]$ and $x[n]$ is the output and input of the matched filter respectively.

2.3.7 Discrete fourier transform

A sampled signal can have its frequency spectrum calculated by a transformation from the time domain to the frequency domain using a discrete fourier transform (DFT) or fast fourier transform (FFT) algorithm. The sampling rate f_s must satisfy certain requirements to minimize aliasing effects [8]. The FFT is a faster and more computationally effective algorithm implementation of the DFT. The FFT divides the DFT computation into smaller and simpler problems. These subproblems are then combined together to make the complete result of the FFT. The most common implementation of the FFT is the Cooley-Tukey algorithm, which requires the dimension N to be a power of two. The FFT is very useful and commonly used to extract the frequency content of a digital signal.

2.3.8 Decimation

Decimation or downsampling is a process that reduces the sampling rate, which is an important concept in DSP, since it effectively reduces the computational load on the system. It also reduces the bandwidth of the signal. Decimation also allows for matching different sample rates in systems with each other. A time discrete signal $x(n)$ with sample frequency f_s can be decimated [8] with a factor L according to equation (2.20).

$$x_{\text{down}}(n) = x(nL), \quad (2.20)$$

where every L sample of the higher frequency signal $x(n)$ is kept. Decreasing the sample rate with a factor L has an effect on the frequency spectrum. The Nyquist interval $[-f_s/2, f_s/2]$ is shrunk by a factor L and frequency components outside of the new decimated Nyquist interval $[-f_s/(2L), f_s/(2L)]$ will cause aliasing. This aliasing can be avoided by using a low pass filter before the decimation [8] which removes the frequency components outside of the decimated Nyquist interval. Decimation will thus reduce the bandwidth of the digital signal, and also reduce the quantization noise of the ADC which leads to lowering of the noise floor.

2.3.9 Digital down conversion

The process of digital down conversion (DDC) is to prepare the data coming from the ADC for digital signal processing [4]. This includes

- Decimation of the sampling rate
- Filtering to increase signal-to-noise ratio (SNR)
- Quadrature mixing of the relevant frequency band into the baseband

The DDC is performed inside the SDR and can be seen in the architecture of the Air-SpyHF+ in figure 3.1.

2.3.10 Modulation

Data can be sent or carried by a RF signal through the process of modulation. A simple unmodulated RF carrier can be described by equation (2.21).

$$V(t) = A \sin(2\pi ft + \phi). \quad (2.21)$$

A is the amplitude, f is the frequency, t is the time and ϕ the phase shift.

For frequency modulation (FM) the information or data is represented by changes in the frequency over time. In this case equation (2.21) can be written as:

$$V(t) = A \sin(2\pi f(t)t + \phi). \quad (2.22)$$

Amplitude modulation (AM) and phase modulation (PM) can be achieved in the same manner [3].

2.3.10.1 On-off keying

On-off keying (OOK) is the simplest form of amplitude-shift keying (ASK) where the carrier is simply turned on or off. This modulation is a form of AM. The presence of a carrier is represented as a binary "1" which is set to transmit for a fixed length of T seconds [4].

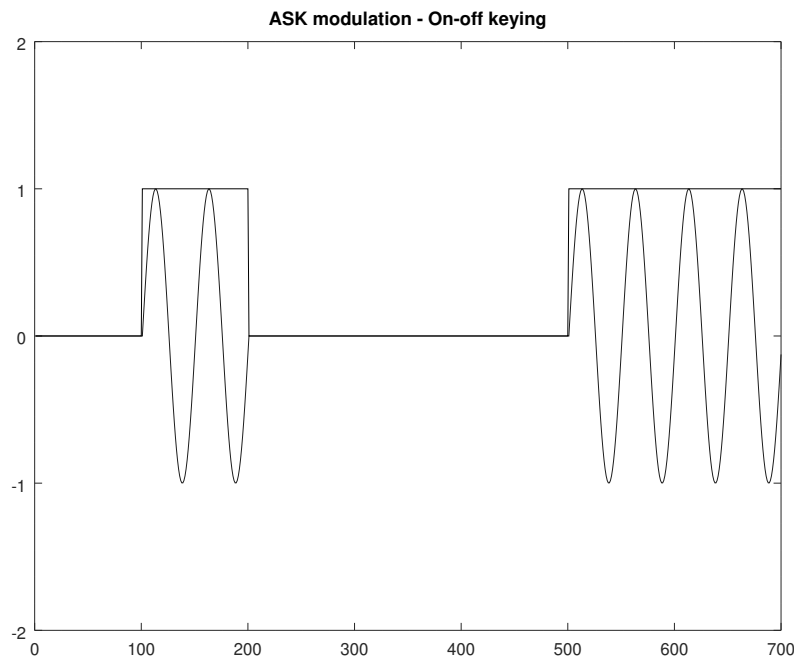


Figure 2.9: On-off keying where a carrier is multiplied with a logical sequence of binary data to be modulated. Every bit is 100 samples. The modulated message is "0100011".

2.3.10.2 IQ data and modulation

IQ data, also synonym to quadrature signals, is commonly used to represent RF signals. Two periodic signals are quadrature to each other if their respective phase differs with 90 degrees ($\pi/2$ radians). The I and Q components are called "in-phase" and "in-quadrature" respectively and can be used as an IQ modulator which can create AM, FM and PM by simply summing amplitude modulated IQ signals [3]. The Q-part is shifted 90 degrees relative to the I-part. A band pass signal $s(t)$ can be represented in the following form:

$$s(t) = I(t) \cos(2\pi f_c t) - Q(t) \sin(2\pi f_c t), \quad (2.23)$$

where $I(t)$ and $Q(t)$ are the amplitude modulated IQ signals and f_c is the carrier frequency.

Amplitude and power calculations from IQ data can be performed in the following way: The RF peak voltage V_p is

$$V_p = \sqrt{I^2 + Q^2}. \quad (2.24)$$

The RMS value of the RF peak voltage is

$$V_{\text{RMS}} = \frac{V_p}{\sqrt{2}} = \frac{\sqrt{I^2 + Q^2}}{\sqrt{2}}. \quad (2.25)$$

The power in a 50Ω system can be calculated according to

$$P_{\text{RMS}} = \frac{V_{\text{RMS}}^2}{R} = \frac{V_{\text{RMS}}^2}{50} = \frac{I^2 + Q^2}{100} \quad [\text{W}]. \quad (2.26)$$

Which in dBm is

$$P_{\text{RMS}} = 10 \cdot \log_{10}[10(I^2 + Q^2)] \quad [\text{dBm}]. \quad (2.27)$$

2.4 Noise

Noise is present everywhere and can impact a radio receiver from internal sources generated within the system itself as well as external sources. The noise sets a limit on the minimum signal strength that can be detected by the system. In order to achieve the best performance it is generally desirable to minimize the noise. Internal noise generated within the system or device is usually caused by random motion of charges or charge carriers in materials and devices. One example of internal noise is thermal noise which is called Johnson and Nyquist noise, and is caused by thermal vibration of bound charges. Some examples of external RF noise are thermal noise from the ground, electromagnetic interference (EMI), wireless devices and radio stations [2].

Possible EMI coming from the UAV motors is of particular interest for this project since it could very well pollute near 457 kHz.

2.5 Serial communication

Serial communication is widely used to transfer data between hardware. In this project serial communication is utilized between the Odroid XU4 and the UAV-system which is shown in greater detail in section 2.5.1.

2.5.1 UART

A Universal Asynchronous Receiver Transmitter (UART) is a physical circuit located inside an integrated circuit (IC) or embedded inside a microcontroller. The purpose of UART is to transmit and receive serial data. Two wires are needed for the communication, one transmitter wire (Tx) and one receiver wire (Rx). The fact that the UART is asynchronous simply means that no clock signal is used to synchronize bits to be transmitted. The UART uses start and stop bits to define the start and end of a packet to be sent or received. One UART data packet contains 1 start bit, 5 to 9 data bits, an optional parity bit 1 or 2 stop bits [7]. The UART data packet is shown in figure 2.10.

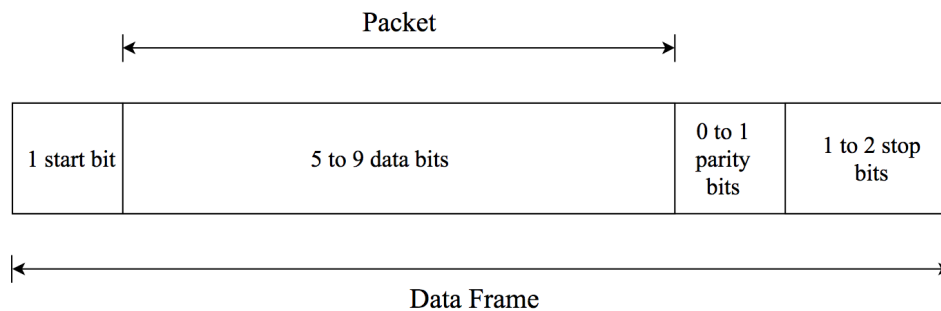


Figure 2.10: Contents of a UART data frame [7].

2.5.2 Mavlink protocol

Mavlink is a communication protocol for communicating with UAVs (drones), and also in between on-board hardware peripherals. In this project the Mavlink protocol is used to transfer the data from the on-board computer to the UAV-system. Mavlink has support for several programming languages, here Python is used. Some of the key features [9] of Mavlink are:

- Efficiency. There are only 8 bytes of overhead per packet.
- Reliability. Providing methods for detecting packet corruption, drops and authentication. Mavlink is used to communicate between different UAVs, ground stations over different channel characteristics with high latency and noise.
- Supports several programming languages and can run on many different microcontrollers and operating systems.
- Allows 255 simultaneous systems on one network. UAVs, ground stations, nodes, etc.

In table 2.1 the packet structure of the Mavlink protocol is shown.

Field name	Index (Bytes)	Purpose
Start of frame	0	Indicates the start of transmission
Payload length	1	Length of the payload (n)
Packet sequence	2	Enabling packet loss detection
System ID	3	System identification
Component ID	4	Component identification
Message ID	5	Message identification
Payload	6 to $(n + 6)$	Message data
CRC	$(n + 7)$ to $(n + 8)$	Checksum of the entire packet

Table 2.1: Packet structure of the Mavlink protocol [9].

2.6 Ferrite rod antenna

A ferrite rod antenna is simply a rod constructed of a ferrite material with wire windings around it. An illustration of its radiation pattern is shown in figure 2.11. This type of antenna is also called a small loop antenna [4]. The major benefit of the ferrite rod antenna is that it can be used for lower frequencies and still be small in size. Common uses are the AM broadcast-band, direction finding equipment and low frequency receiving systems below 500 kHz. It's a magnetic antenna since it couples the magnetic field of a radio wave in the near field of the antenna. The reactive near field region R can be found by using equation (2.28).

$$R < \lambda/2\pi, \quad (2.28)$$

where λ is the wavelength of the RF signal. For a frequency of 457 kHz, the reactive near field region becomes approximately $R < 103$ m. The magnetic field strength in the reactive near field region is proportional to the inverse-cube [15] of the distance $1/R^3$, which in terms of power decays at a rate of $1/R^6$. Figure 2.11 shows the radiation pattern of a ferrite rod antenna.

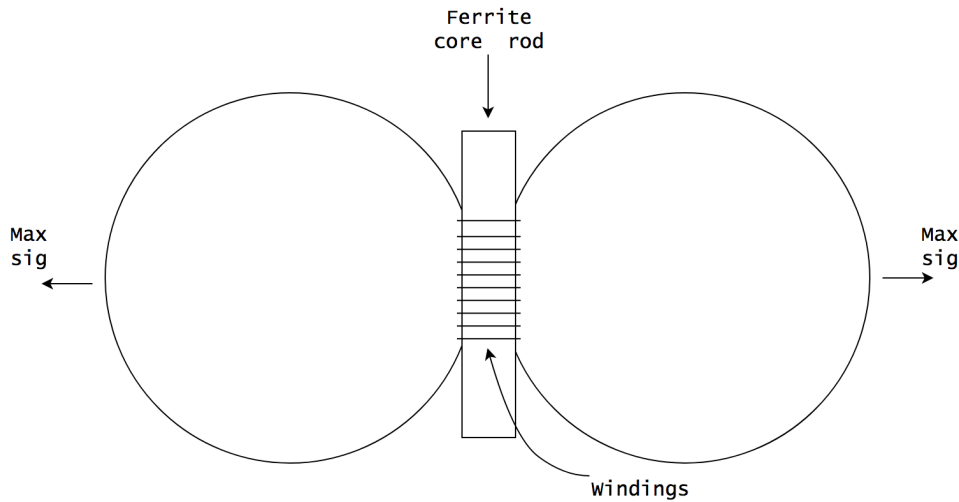


Figure 2.11: Radiation pattern of a ferrite rod antenna.

2.7 Link budget

When calculating a link budget the common contributions [2] to the link are the following:

- Transmit power
- Transmit antenna line loss
- Transmit antenna gain
- Path loss (free-space)
- Atmospheric attenuation
- EMI noise addition, making the dynamic range worse.
- Receiver antenna gain
- Receiver antenna line loss

The transmit power, antenna gain and antenna line loss of the avalanche beacon are unknown parameters that depends on how the avalanche beacon is constructed. The most critical parameters in the link budget will be the path loss and EMI which will have the most impact on the system performance. The path loss is proportional to the inverse-cube of the the distance, mentioned in section 2.6. The EMI will increase the noise floor and reduce the dynamic range of the radio receiver. In an office environment for instance, the EMI is substantially higher than outside in a free mountain-like environment.

3 Hardware

Various types of hardware was needed to develop the radio receiver system. Apart from the necessary hardware for the radio receiver itself some test equipment was needed.

Throughout the development process a lab bench was used to imitate the behavior of a transmitting avalanche beacon. For the field-tests a real avalanche beacon and the actual antennas were used.

3.1 Radio receiver hardware

3.1.1 SDR: AirSpy HF+

With regards to the systems requirements the most important part of hardware is the SDR, that needs to be able to detect weak signals at the given frequency of 457 kHz. The AirSpy HF+ was considered sufficient because of its MDS of -140 dBm [5] and correct frequency range using the HF RF input port. Two other important aspects are that it streams the IQ data over USB and has support for GNU Radio. According to equation (2.6), the 16 bit ADC in the AirSpy HF+ will result in a dynamic range of $6 \cdot 16 = 96$ dB. The reasonable price point at approximately 2500 SEK is also an important factor. The relevant specifications [5] of the AirSpy HF+ are:

- HF coverage between 9kHz and 31 MHz.
- -140 dBm ($0.02 \mu\text{V}/50 \text{ ohms}$ at 15 MHz) MDS (minimum detectable signal).
- Up to 660 kHz alias and image free output for 768 ksps IQ data over USB.
- 18 bit DDC, scaled to 16 bit.
- +10dBm maximum RF input.
- 2x High Dynamic Range Sigma Delta ADCs at up to 36 MSPS.
- RF inputs matched to 50 ohms.

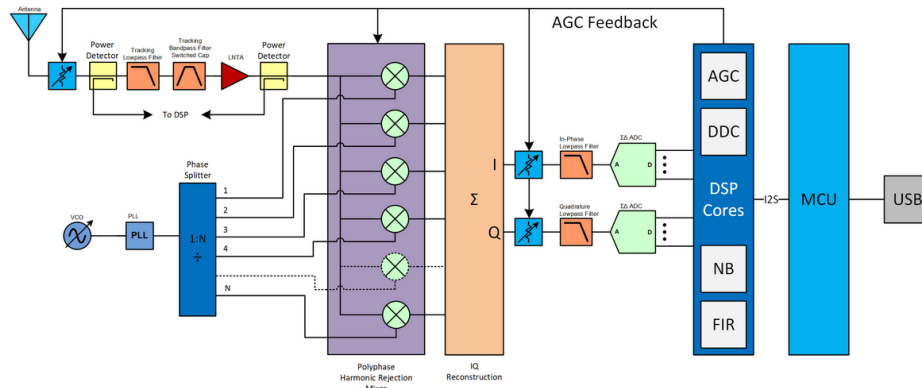


Figure 3.1: Internal structure of the AirSpy HF+ [5].

Figure 3.1 shows how the analog signal is processed and digitized in the AirSpy HF+. The output of the SDR is raw IQ-data streamed over USB at a rate of 768 kbps which can be further processed digitally using for instance GNU Radio. Figure 3.2 shows an overview of the AirSpy HF+ with the PCB, the two RF inputs on the left and USB output on the right. The bottom RF input is used for the purposes of this project since it covers the right frequency range.

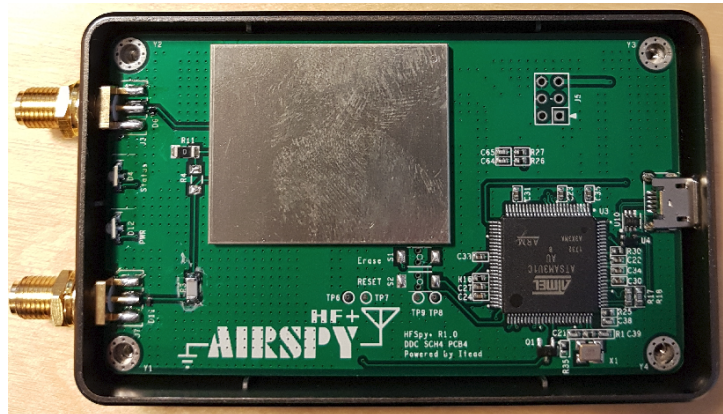


Figure 3.2: Top view of the AirSpy HF+ with its metal casing removed.

3.1.2 Single-board computer: Odroid XU4

The avalanche beacon signal could appear in the frequency range $457 \text{ kHz} \pm 80 \text{ Hz}$, therefore one needs to adjust the DSP-chain to take the entire band of 160 Hz into account [14]. The fact that it is a very narrow band one can decimate the signal substantially and thereby reduce the computational power required by the single-board computer. Due to this fact the DSP wont be very demanding for the CPU on the single-board computer. Although, a relatively powerful single-board computer was still chosen because there might be further improvements on the DSP and software in the future work of this project that requires greater computational power. A part from the computational power, the Odroid XU4 single-board computer was chosen for its many peripherals, GPIO and support for serial communication. It can also run Linux-based operating systems which is a requirement for using GNU Radio. Figure 3.3 shows the Odroid XU4 used in this thesis.

The GPIO pins on the Odroid XU4 runs from 3.3V. Therefore a XU4 Shifter Shield was used, which is a separate circuit board that mounts on top of the single-board computer to convert the GPIO pins to 5V. The relevant specifications [6] of the Odroid XU4 are the following:

- CPU: ARM Cortex A15 @ 2.0GHz / ARM Cortex A7 @ 1.4GHz.
- RAM: 2GB DDR3.
- I/O: 2x USB 3.0 ports, 1x USB 2.0 port, HDMI, ethernet, GPIO pins.
- GPIO serial communication: UART, SPI and I2C.
- Can run Linux-based operating systems.

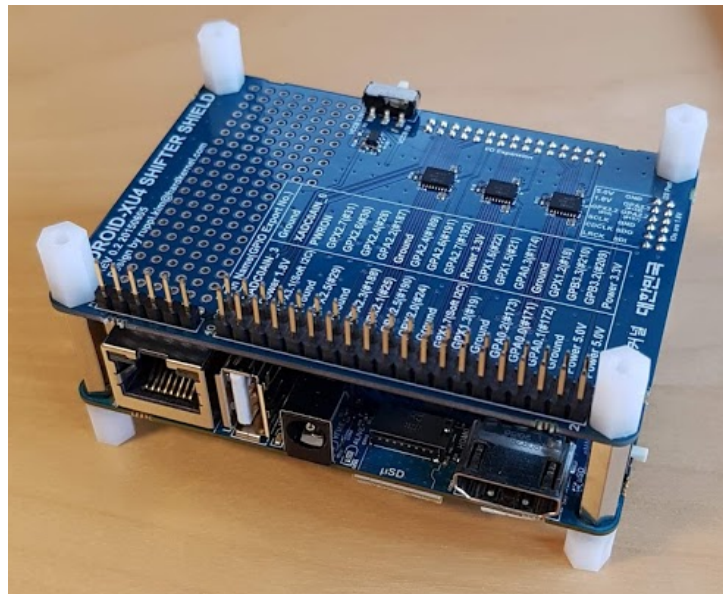


Figure 3.3: The Odroid XU4 used in this project with the XU4 Shifter Shield mounted on top.

3.1.3 Ferrite rod antennas and LNA

For this project two ferrite rod antenna was used. They were provided from experts within the larger project. Along with the antennas, a resonance and matching circuit was implemented to ensure that the antennas receive at the correct frequency of 457kHz. It also ensures an input impedance of 50 ohm. A schematic for this resonance and matching circuit can be found in figure 3.4. Figures 3.5 and 3.6 shows the frequency characteristics of the LNA in terms of noise and passband. This is the LNA mentioned in the project description. The actual antennas used for the field test measurements can be seen in figure 3.7.

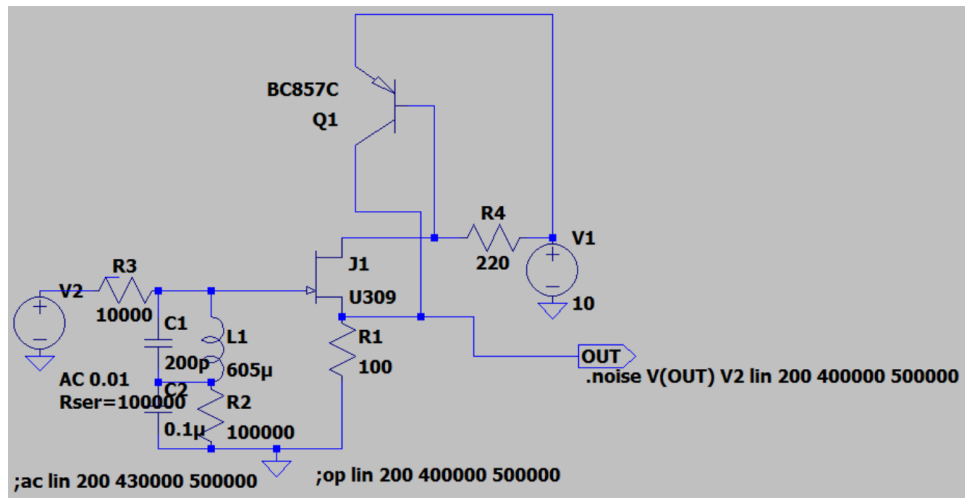


Figure 3.4: LTSpice simulation of the resonance and matching circuit. L1 being the antenna.

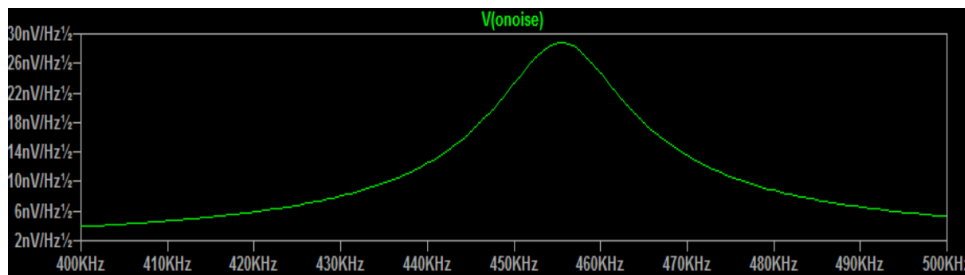


Figure 3.5: Thermal and transistor noise of the circuit in figure 3.4.

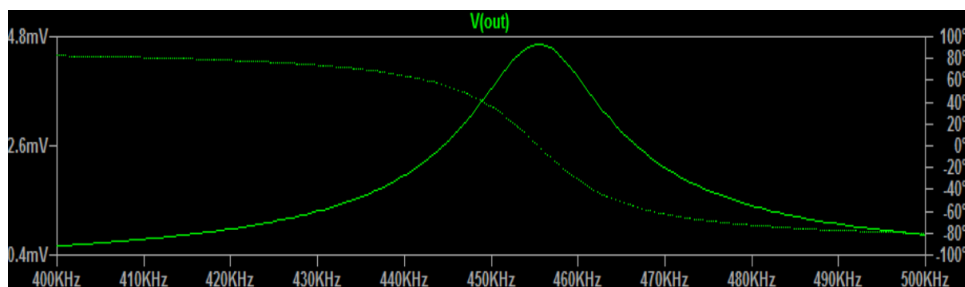


Figure 3.6: Passband of of the circuit in figure 3.4.

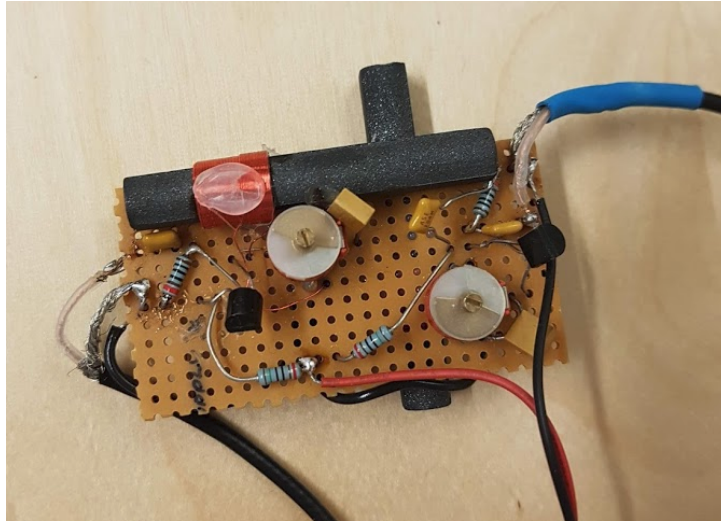


Figure 3.7: Two ferrite rod antennas mounted perpendicular to each other with the matching resonance circuit from figure 3.4.

3.2 Test equipment

3.2.1 Signal generator

A signal generator was used to imitate the avalanche beacon, here the Rohde Schwarz SME06, which supports RF output of 5kHz to 6GHz, was used. The 50 ohm RF output was used and the settings for the signal was:

- Modulation: Pulse-gen. Constant output was also used for testing.
- Frequency: 457 kHz $\pm$ 100 Hz.
- Period: 800 ms.
- Width: 100 ms.

The settings above imitates the characteristics of an avalanche beacon mentioned in section 2.1.

3.2.2 Logic analyzer

In order to verify the functionality of the implemented UART communication from the single-board computer to the UAV-system a logic analyzer was used. Logic analyzers are good tools for debugging and verifying serial communication because of their ability to trace and display logical signals during operation.

3.2.3 Avalanche beacon

The avalanche beacon used for the field-tests was a unit from Barryvox, which is illustrated in figure 3.8. Initial measurements and testing of this particular avalanche beacon showed that it had an on-time of 100 ms.

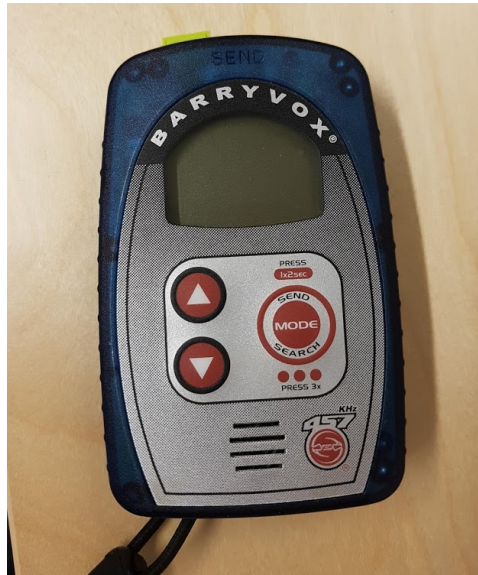


Figure 3.8: Barryvox avalanche beacon. It contains both a searching and a transmitting mode. This unit was used in the field test of the radio receiver.

4 Software

Various software tools were used during the development of the radio receiver. The digital signal processing chains (DSP-chains) were implemented in GNU Radio. The external software for detection decision making and data transfer to the UAV-system was developed in Python. Post-processing of data for plots and figures was performed in MATLAB and Octave.

4.1 Linux

The operating system running on the single-board computer is a light version of Linux Ubuntu called Mate. Linux supports the use of GNU Radio, and makes interacting with hardware more convenient. For instance when setting up the UART-communication between the single-board computer and the UAV-system.

4.2 GNU Radio

GNU Radio is a free and open-source software development toolkit that contains signal processing functions which can be used to implement a custom software radio. The GNU Radio framework is written in C++ and it can be used purely as a simulation-like environment without hardware, or used to create a software defined radio with readily-available hardware like the AirSpy HF+. An application created with GNU Radio is usually referred to as a "flowgraph" which has a block like structure when running the GUI version called GNU Radio Companion (GRC). The flowgraphs themselves can be developed in Python or C++.

In order to get the streamed IQ data from the AirSpy HF+ to GNU Radio one needs to use the custom GNU Radio blocks provided by grOsmoSDR [13]. This block enables the streaming of IQ data over USB and also allows for tweaking parameters in the firmware of the AirSpy HF+. For example: setting internal gains, toggling automatic gain control (AGC) among other settings. For this project the AGC was turned off, and the internal gain set to a maximum of +6 dB. The reason for turning off the AGC is that we want full control of the measured signal strength, because we are going to calculate the angle towards a transmitting avalanche beacon based on the ratio between the two measured signal strengths from the two antennas.

4.3 ZeroMQ

ZeroMQ is an open-source high-performance asynchronous communications library. It contains an *API* that provides a socket type structure of communication where one can for instance use a push-socket in one end, and a pull-socket in the other. This push-pull feature is called a *pattern* and there are several other patterns included in the *API*. This project only makes use of the push-pull pattern which works as a pipeline where the information is "instantly" sent through the socket when it is available at the push-socket [11]. ZeroMQ is supported in GNU Radio and is used in this project to transfer the data in real-time from GNU Radio to an external Python-script taking care of the detection algorithm and the data transfer to the UAV-system.

5 Implementation

The implementation and development process of the radio receiver was roughly performed in the following steps:

- Literature study on ETSI EN 300 718-1, DSP, SDR and GNU Radio.
- Hardware analysis and review of different hardware possibilities for single-board computer and SDR.
- Ordering hardware.
- Getting the hardware (SDR and single-board computer) to interact with GNU Radio.
- Implementation and testing of three different DSP-chains.
- Research a way to transfer data from GNU Radio to an external program (ZeroMQ).
- Integration of ZeroMQ with GNU Radio and Python.
- Implementation of the serial communication from the single-board computer to the UAV-system.
- Lab bench test of the radio receiver.
- Field test of the radio receiver with the best performing DSP-chain.

The implementation of the DSP assumes that the avalanche beacons satisfy ETSI EN 300 718-1, thus the signal processing and detection algorithm can be customized to detect signals using this standard.

5.1 DSP-chain

Three different DSP-chains were implemented in this project. The first one utilizes the FFT, the second one runs the signal through a bank of band pass filters, and the third approach uses a matched filter. Two parallel DSP-chains are required in all three approaches because the project uses two antennas and two SDRs.

5.1.1 FFT

This DSP approach consists of the following parts:

1. Receiving IQ data from the AirSpy HF+ at 768 ksps [5].
2. FIR low pass filter and decimate.
3. Perform FFT.
4. Convert to power.
5. Transfer data to Python script.

The AirSpy HF+ is set to receive at the center frequency of 457 kHz. The default sampling rate of 768 kbps is an unnecessarily high data rate with too large bandwidth and is thereby decimated. The decimation factor is set to 3200, which reduces the sampling rate and bandwidth to 240 Hz around the center frequency. The FIR low pass filter is used to remove aliasing that would otherwise arise due to decimation, as previously mentioned in section 2.3.8. The FIR low pass filter parameters are:

- Gain = 1.
- Cut-off frequency = 120 Hz (center frequency at 457 kHz.)
- Transition bandwidth = 60 Hz.
- Window function = Hamming window, $\beta = 6.76$.
- Decimation factor = 3200.

In GNU Radio the FIR low pass filtering and decimation was performed using a block called "Xlating FIR Filter", where the coefficients were calculated using the existing GNU Radio filter library functions. The Hamming window was chosen for its balanced properties in the frequency domain. It has good enough attenuation of side-lobes and the main lobe is not too wide. The properties of the Hamming window is shown in section 2.3.5.

The FFT size was chosen to be 8. With a bandwidth of 240 Hz this results in 30 Hz per frequency bin. The expected avalanche beacon signal should then appear in the six most center bins. If the bins are numbered from 1 to 8, the bins containing the signal would be 2,3,4,5,6 and 7 containing the band of $457 \text{ kHz} \pm 90 \text{ Hz}$. This FFT vector of size 8 is sent to an external Python script as soon it is available using ZeroMQ. The DSP is intentionally performed so that every frequency bin is 30 Hz which is 33.33 ms in the time domain. Remembering that the pulse shape of the avalanche beacon in section 2.1, is greater or equal to 70 ms, one cannot have frequency bins that are longer than $70/2 = 35 \text{ ms}$ in the time domain, because one might simply miss parts of the avalanche pulse.

In the external Python script averaging is performed to buffer in at least 100 ms of data. Also it is the matter of setting a power threshold in dBm where a detected signal strength is above this particular threshold. Initial testing and measurements showed that the particular avalanche beacon used in this project (see section 3.2.3) had an on-time of 100 ms.

The detected signal strength above the threshold is sent over UART using the Mavlink protocol along with a timestamp of the measurement and an angle calculated based on the ratio between the signal strengths measured in each antenna.

An overview of the signal processing using the FFT approach can be seen in figure 5.1.

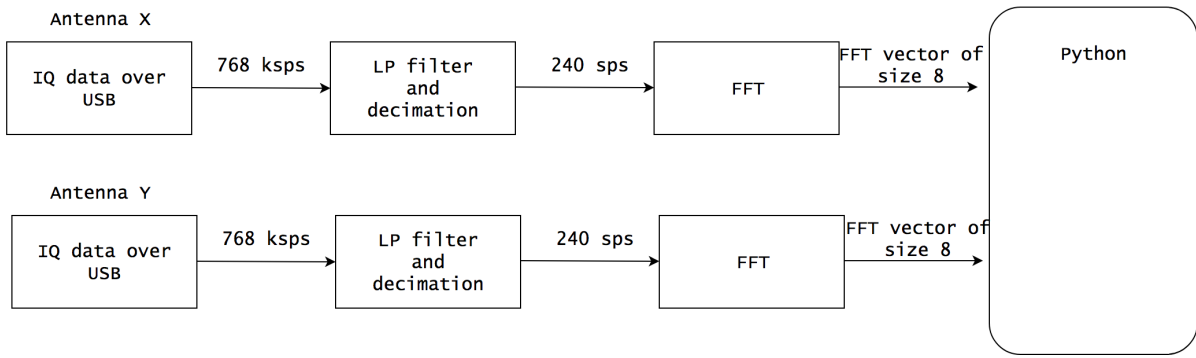


Figure 5.1: DSP-chain utilizing the FFT.

5.1.2 Filter bank

This approach also starts with a FIR low pass filter and decimation, but instead of taking the FFT a bank of narrow band pass filters are used to detect the avalanche beacon signal.

1. Receiving IQ data from the AirSpy HF+ at 768 ksps [5].
2. FIR low pass filter and decimate.
3. Eleven narrow band pass filters forming a bank.
4. Package data using a mux.
5. Transfer data to Python script.

In this case the decimation factor is set to 1600, reducing the sampling rate and bandwidth of the IQ data to 480 Hz. The FIR low pass anti aliasing filter has the following parameters:

- Gain = 1.
- Cut-off frequency = 240 Hz (center frequency at 457 kHz.)
- Transition bandwidth = 120 Hz.
- Window function = Hamming window, $\beta = 6.76$.
- Decimation factor = 1600.

The narrow band pass filters in the bank are set to have a pass band of 40 Hz. They are also set to overlap each other with 20 Hz in the passband making sure that there are no "gaps" where the possible avalanche beacon signal could be attenuated. Then eleven band pass filters cover the frequency band of $457 \text{ kHz} \pm 100 \text{ Hz}$, thus also the avalanche beacon signal. There are eleven parallel channels for each antenna in this case, because of the filter bank. A decimation of a factor 16 is also performed in each band pass filter to reduce the sample rate and bandwidth further.

The eleven band pass filter with their parameters are the following (except for the high and low cut-off frequencies that are adjusted for each separate filter):

- Gain = 1.

- High cut-off frequency = 20 Hz (for the center most BP filter).
- Low cut-off frequency = -20 Hz (for the center most BP filter).
- Transition bandwidth = 10 Hz.
- Decimation factor = 16.
- Window function = Hamming window, $\beta = 6.76$.

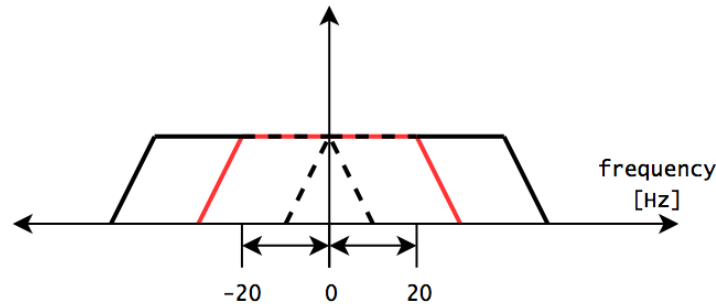


Figure 5.2: Three overlapping band pass filters. Each with a pass band of 40 Hz and a transition width of 10 Hz. The red colored filter being the center BP filter with a 40 Hz pass band between -20 and +20 Hz.

An illustration of a simpler implementation using three overlapping band pass filters is shown in figure 5.2. The band pass filters reduce the sampling rate with a factor of 16 making the sampling rate and bandwidth 30 Hz for each filter. In the time domain this means that one sample is 33.33 ms.

After the band pass filter bank the eleven channels are sent to a mux which orders the data into a vector that can be sent to Python. This is done to avoid having eleven parallel channels into Python and instead getting only one channel, with a vector containing the data from the eleven channels. In the Python script the detection algorithm looks for a signal power level above a set power threshold. This approach also buffer in 100 ms of data (three samples) to ensure that one sample is completely within a pulse of length 100 ms.

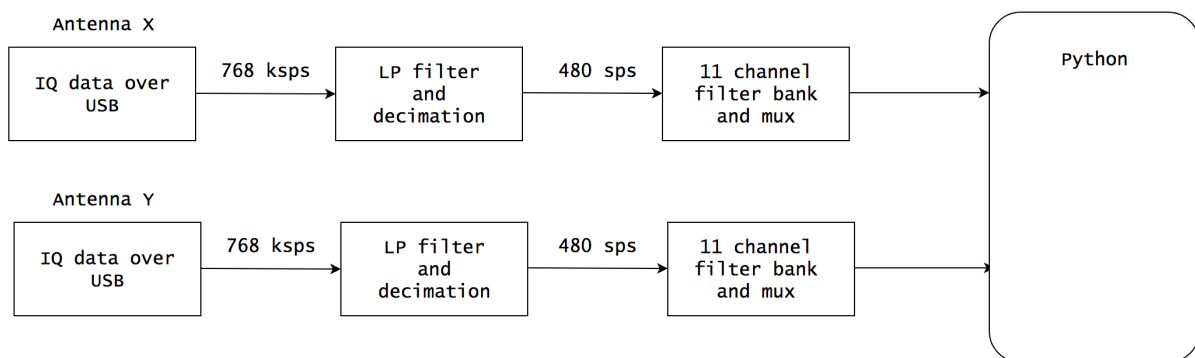


Figure 5.3: DSP-chain using bank of band pass filters.

An illustration of the filter bank DSP-chain can be seen in figure 5.3. The filter bank

implementation can also show in which of the eleven band pass filters the avalanche beacon signal was detected.

5.1.3 Matched filter

The matched filter DSP-chain contains the following:

1. Receiving IQ data from the AirSpy HF+ at 768 ksps [5].
2. FIR low pass filter and decimate.
3. Cross-correlating the IQ data with a template of the avalanche beacon pulse making the matched filter.
4. Transfer data to Python script.

The FIR low pass anti aliasing filter has the following parameters:

- Gain = 1.
- Cut-off frequency = 240 Hz (center frequency at 457 kHz.)
- Transition bandwidth = 120 Hz.
- Window function = Hamming window, $\beta = 6.76$.
- Decimation factor = 1600.

This results in a sampling rate and bandwidth of 480 Hz. The IQ data is then sent through a matching filter. In particular a FIR filter with its coefficients being a template of the avalanche beacon pulse according to the theory in section 2.3.6. The template pulse is set to have an on-time of 100 ms, which is 48 samples, with a sampling rate of 480 sps. If $T_s = 48$ is the number of samples per pulse.

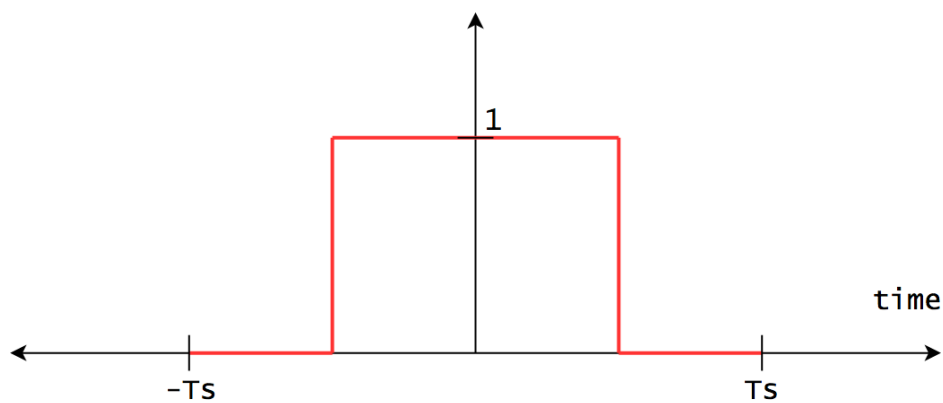


Figure 5.4: Pulse template where the on-time (binary 1) is 48 samples = 100 ms.

The total number of taps of the matched filter is $48 \cdot 2 = 96$ according to figure 5.4. The output of the matched filter is then transferred to python 480 samples at a time making up one second of data.

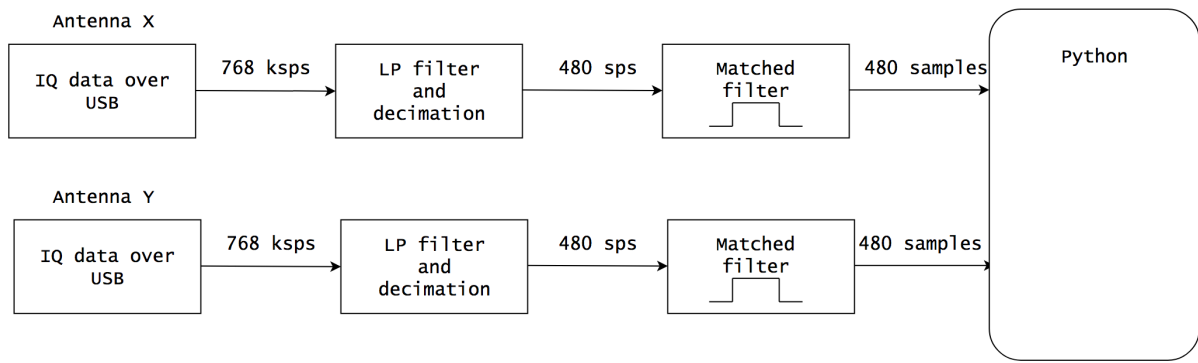


Figure 5.5: DSP-chain using a matched filter with a template of the avalanche beacon pulse form.

5.2 Software written in Python

The key features of the software developed in Python is the detection decision making process, the interaction with GNU Radio using ZeroMQ and the UART communication to the UAV-system. As for the matched filter approach the software has the following structure:

1. Import necessary libraries (numpy, struct, zmq, time and pymavlink).
2. Initialize the ZeroMQ connection to GNU Radio.
3. Initialize the UART connection to the UAV-system.
4. Declare necessary variables (power threshold and calibration factor).
5. Enter the main while loop.
6. Wait for new incoming data from GNU Radio from both antennas. New data every second.
7. Square the data and look for the maximum value.
8. Calculate the power in dBm.
9. If the maximum value in dBm is above the power threshold it is considered to be a detection. Then transfer the signal strengths in dBm for each antenna along with a time stamp of measurement.
10. Go back to the top of the main while loop and wait for new incoming data.

The software written for the FFT and filter bank methods follow similarly. The code can be found in Appendix.

ZeroMQ and the UART communication using the Mavlink protocol are initialized the following way:

```
# Necessary libraries
import zmq
from pymavlink import mavutil

# Initialization of the ZeroMQ connection with GNU Radio
context = zmq.Context()

# Two sockets, one channel for each SDR (AirSpy HF+)
socketX = context.socket(zmq.PULL)
socketY = context.socket(zmq.PULL)

# The ZeroMQ push sink in GNU Radio is tied to the same TCP
↪ addresses as here.
socketX.connect("tcp://127.0.0.1:5555")
socketY.connect("tcp://127.0.0.1:5556")

# Connection to the UART/Serial bus of the Odroid XU4
# Baud rate set to 115200 bits/sec
the_connection = mavutil.mavlink_connection('/dev/ttySAC0', 115200)
```

Data from the ZeroMQ sockets is received according to:

```
dataX = socketX.recv()
dataY = socketY.recv()
```

Sending a Mavlink message over UART is performed by:

```
the_connection.mav.debug_vect_send('Detection', time_usec,
    ↪ x_sig_str_dBm, y_sig_str_dBm, angle)
```

5.3 System tests

Two types of tests were performed during the development. The Lab bench test, involving the use of a signal generator, and a field test using the actual antennas and an avalanche beacon.

5.3.1 Lab bench test

The lab bench test was setup with the signal generator mentioned in section 3.2.1. This system test allowed for convenient switching of the input RF signal frequency, power level and shape. Also immediate changes could be done to the DSP. The signal generator was connected directly to the two SDRs which in turn was connected to a PC running the DSP in GNU Radio and the external Python script for easy output of data to a terminal window. This was how the majority of testing was performed and the how the dynamic range of the different implementations of the system was measured.

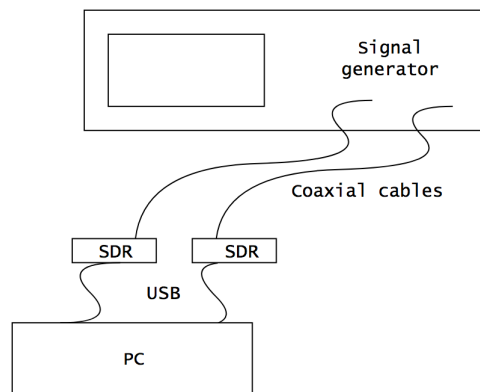


Figure 5.6: Lab bench test setup.

5.3.2 Field test

The field tests with the actual antennas and an avalanche beacon were performed inside the offices at ÅF Digital Solutions. Unfortunately no outside or antenna chamber tests were performed. The field test measured the maximum range of a successful detection for each of the three DSP-chains.

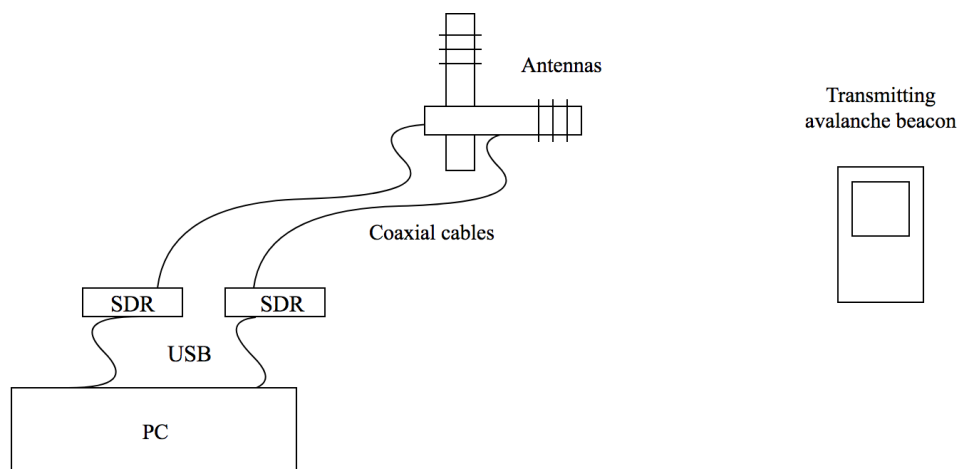


Figure 5.7: Field test setup.

6 Results

This section will contain the results from the tests performed on the developed radio receiver. The results from the matched filter approach will be shown first, because this DSP-chain gave the best performance. However, the three DSP-chains were developed in the following order: FFT, filter bank and lastly the matched filter.

When performing the field test measurements it is important to see how the noise floor rises when the antennas are active (supplied with +12V). This noise depends heavily on the environment of where the measurements are taking place. The noise shown in figure 6.1 was recorded at the office of ÅF Digital Solutions in Uppsala and it indicates that the noise floor increases by approximately 35 dB in this particular environment, essentially making dynamic range of the radio receiver worse by 35 dB.

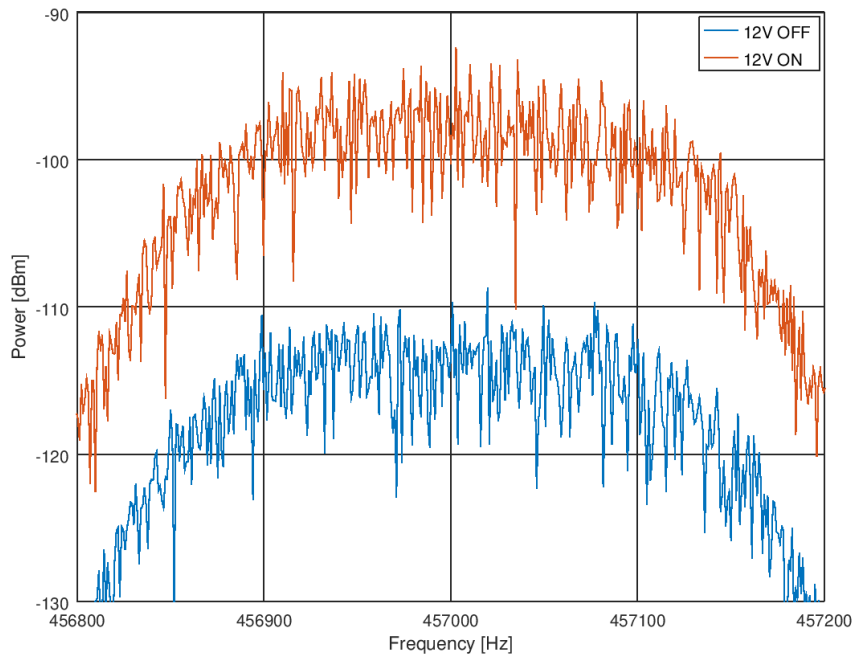


Figure 6.1: FFT of an empty channel with the antennas connected to the SDRs. No avalanche beacon transmitting.

6.1 Matched filter

6.1.1 Lab bench Test

The signal generator settings was set according to section 3.2.1. The lab bench test resulted in a minimum detectable signal level of -135 dBm using the matched filter DSP-chain, thus the power level detection threshold was set to this value. An illustration of this result can be seen in figure 6.2. Presence of a signal level above the power threshold will count as a successful detection of the avalanche beacon. This power level is set in the external Python software. According to [5] the MDS is -140 dBm at a bandwidth of 500 Hz, thus a result of -135 dBm is relatively good i terms of utilizing the maximum available dynamic range of the AirSpy HF+.

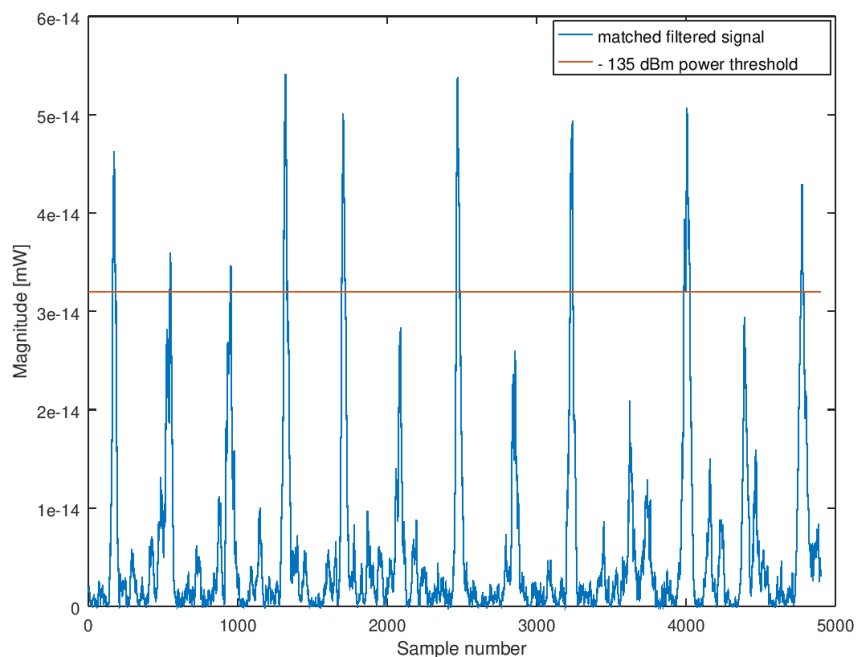


Figure 6.2: Output of the matched filtered DSP-chain. The -135 dBm power threshold is also shown. Sample rate at 480 samples per second. Plot showing just over 10 seconds of data.

6.1.2 Field test

The maximum range field test was setup in the following way where the transmitting avalanche beacon was placed at various distances from the radio receiver. The radio receiver illustrated as the antennas in figure 6.3.

Due to the noisy environment where the field test measurements took place, from figure 6.1 the power threshold for measurements needs to be set 40 dB higher (an additional +5 dB to have some margin). Making the -135 dBm lowest possible power threshold $-135 \text{ dBm} + 40 \text{ dB} = -95 \text{ dBm}$ to avoid false detections.

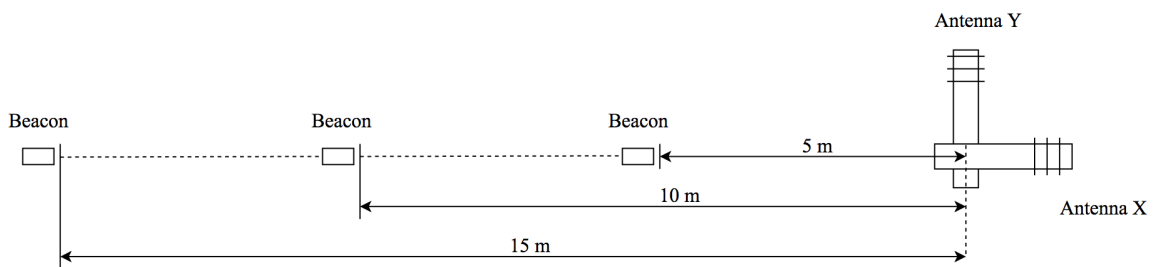


Figure 6.3: Field test setup for the maximum range of detection test.

Firstly the transmitting avalanche beacon was positioned 5 meters from the antennas. In figure 6.4 and 6.5 approximately two seconds of data from this field test setup can be seen. This result shows the avalanche beacon signal before it enters the matched filter. The pulses are approximately 48 samples = 100 ms in time.

Secondly the transmitting avalanche beacon was placed 10 meters from the antennas. Figures 6.6 and 6.7 shows the output of the matched filter. Signal levels above the power threshold is considered a successful detection. The results from the Y antenna shows higher signal levels than the X antenna, which is expected due to the orientation of the antennas (figure 6.3). The X antenna has its null direction towards the beacon.

Thirdly the transmitting avalanche beacon was placed 15 meters from the antennas. Figures 6.8 and 6.9 shows the output of the matched filter at this range. The maximum reliable detection range for the matched filter DSP-chain was approximately 15 meters where the Y antenna manages to detect the beacon. The X antenna however does not.

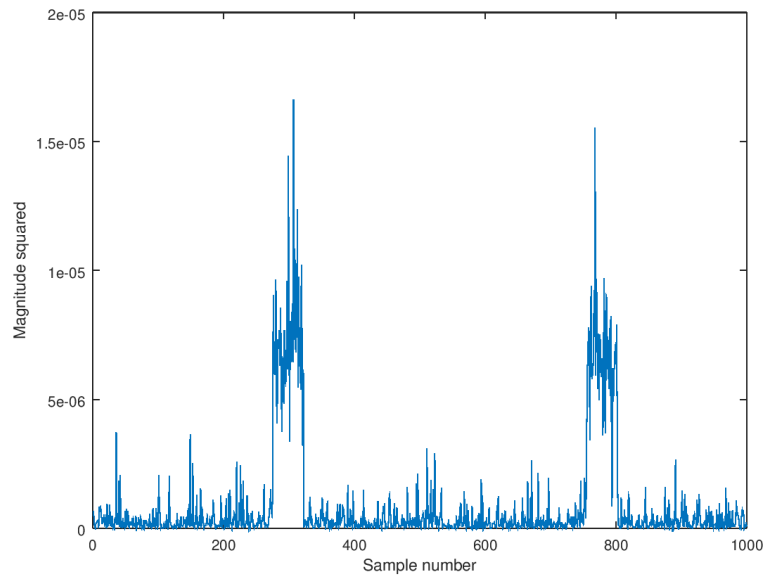


Figure 6.4: Transmitting beacon 5 meters from antennas. Low pass filtered and decimated data by a factor of 1600, reduced sample rate to 480 samples per second. Data from antenna X.

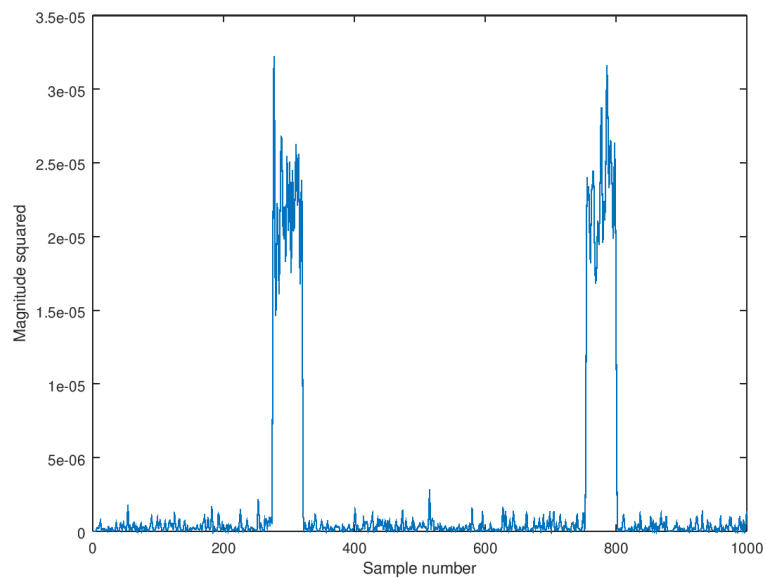


Figure 6.5: Transmitting beacon 5 meters from antennas. Low pass filtered and decimated data by a factor of 1600, reduced sample rate to 480 samples per second. Data from antenna Y.

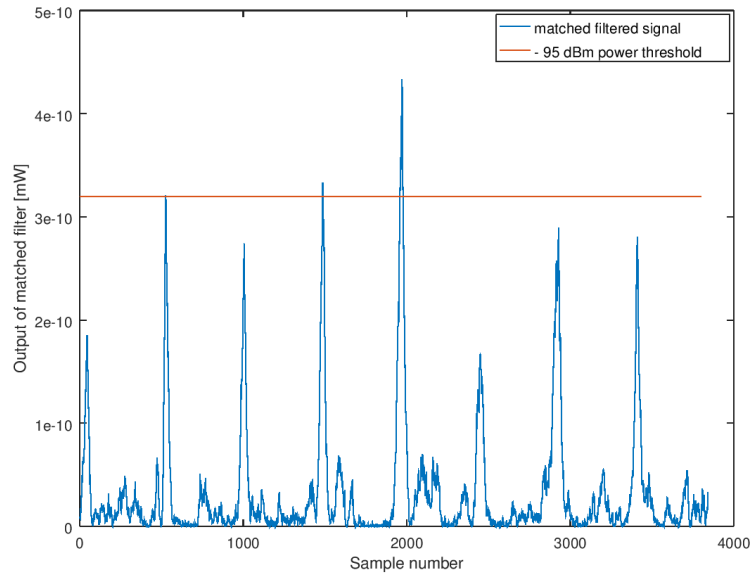


Figure 6.6: Transmitting beacon at 10 meters from antennas. Output of the matched filter scaled to mW. The power threshold set to - 95 dBm. Data from antenna X.

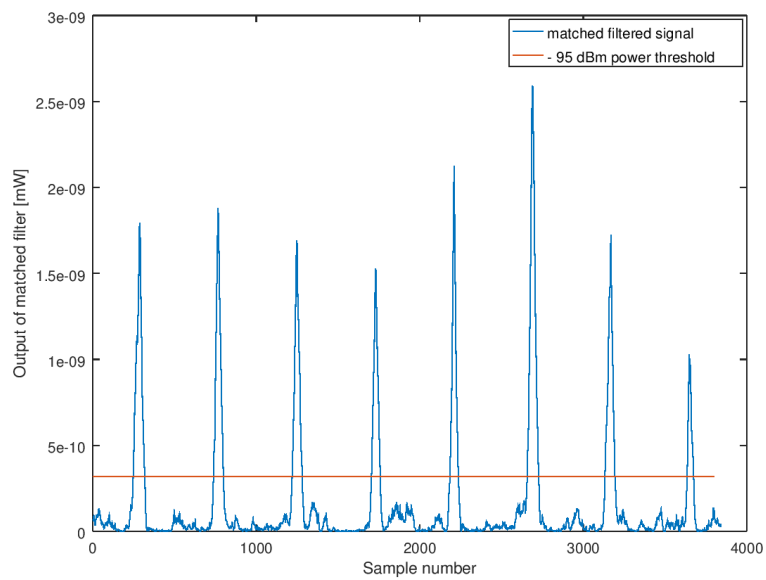


Figure 6.7: Transmitting beacon at 10 meters from antennas. Output of the matched filter scaled to mW. The power threshold set to - 95 dBm. Data from antenna Y.

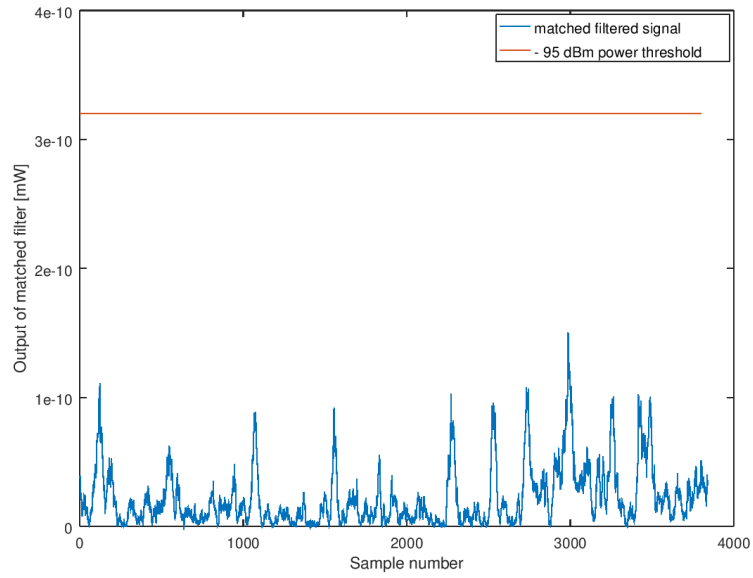


Figure 6.8: Transmitting beacon at 15 meters from antennas. Output of the matched filter scaled to mW. The power threshold set to - 95 dBm. Data from antenna X.

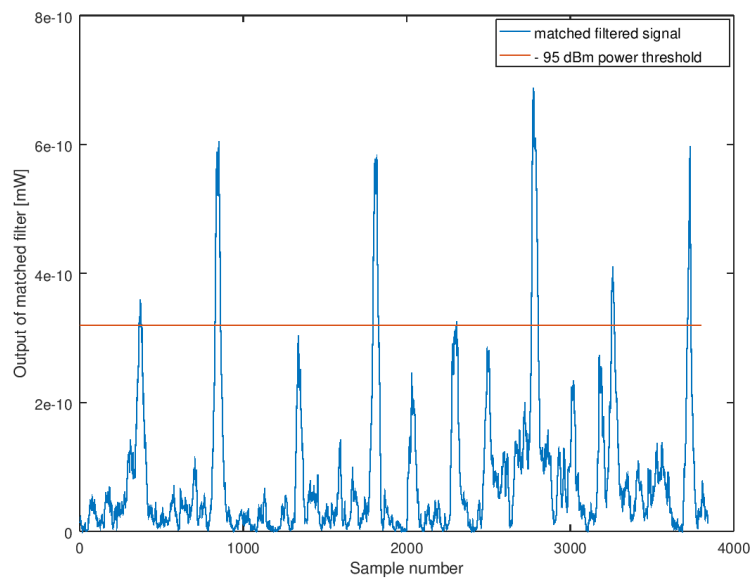


Figure 6.9: Transmitting beacon at 15 meters from antennas. Output of the matched filter scaled to mW. The power threshold set to - 95 dBm. Data from antenna Y.

6.2 Filter bank

6.2.1 Lab bench test

The signal generator settings was set according to section 3.2.1. The lab bench test resulted in a minimum detectable signal level of -125 dBm using the filter bank method, thus the power level detection threshold was set to this value. This result is 10 dB worse than the matched filter approach. For the rest of the lab bench test the RF output power of the signal generator was set to -120 dBm.

The filter bank detection was then tested by sweeping the frequency in the range 457 kHz $\pm$ 100 Hz. The results can be seen in table 6.1. This DSP-chain manages to distinguish in which of filters in the bank the signal was detected.

Input Frequency [Hz]	Filter of detection	Signal power [dBm]
456900 $\pm$ 10	0	-119.8 dBm
456920 $\pm$ 10	1	-120.1 dBm
456940 $\pm$ 10	2	-120.3 dBm
456960 $\pm$ 10	3	-120.2 dBm
456980 $\pm$ 10	4	-119.5 dBm
457000 $\pm$ 10	5	-119.9 dBm
457020 $\pm$ 10	6	-120.0 dBm
457040 $\pm$ 10	7	-119.8 dBm
457060 $\pm$ 10	8	-120.4 dBm
457080 $\pm$ 10	9	-120.1 dBm
457100 $\pm$ 10	10	-119.3 dBm

Table 6.1: Lab bench detection test of the filter bank method. Input RF signal from the signal generator 3.2.1. The frequency was swept in the interval 457 kHz $\pm$ 100 Hz. The same input power of -120 dBm was used for the entire sweep. The signal was pulse formed according to 3.2.1.

6.2.2 Field test

The maximum range field test of the filter bank method followed exactly as the matched filtered approach in section 6.1.2. The result was a maximum detection range of 5 meters using the filter bank method.

Apart from the maximum range field test a full measurement of 360 degrees around the antennas at 2.3 meters was performed using the filter bank method. The measurements of the detected signal strengths were recorded every 30 degrees around the antennas making a total of 12 measurements. The setup of this measurement is illustrated in figure 6.10.

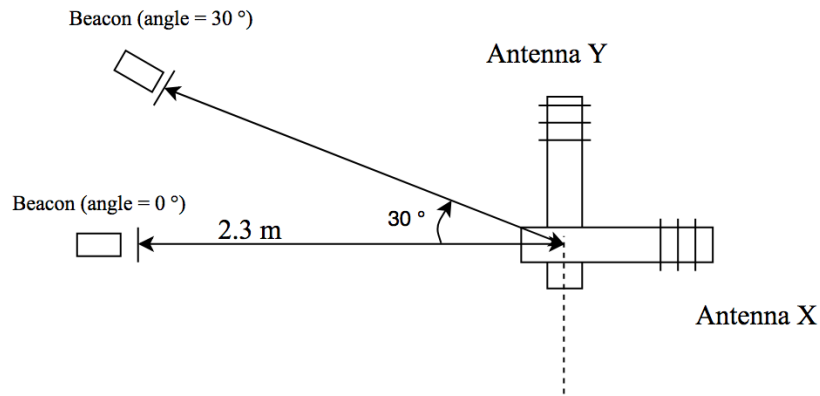


Figure 6.10: Field test setup.

Antenna X power	Antenna Y power	Measurement angle
-68.6 dBm	-68.7 dBm	0 °
-65.2 dBm	-73.6 dBm	30 °
-62.3 dBm	-62.8 dBm	60 °
-61.0 dBm	-55.8 dBm	90 °
-66.3 dBm	-55.8 dBm	120 °
-72.2 dBm	-60.0 dBm	150 °
-66.8 dBm	-64.8 dBm	180 °
-67.8 dBm	-62.3 dBm	210 °
-62.5 dBm	-64.0 dBm	240 °
-63.4 dBm	-58.7 dBm	270 °
-64.1 dBm	-54.4 dBm	300 °
-70.0 dBm	-56.5 dBm	330 °

Table 6.2: Measurement data from the field test setup shown in figure 6.10.

6.3 FFT

6.3.1 Lab bench test

The lab bench test of the FFT approach for the DSP-chain resulted in a minimum detectable signal level of -115 dBm. The testing followed exactly as performed for the matched and filter bank methods. This result is 20 dB worse than the matched filter approach.

6.3.2 Field test

The field test for the FFT approach resulted in a maximum range of detection at approximately 5 meters.

6.4 EMI measurements

For this test the two antennas were placed on approximately 0.5m above the UAV. The UAV was strapped in place and could throttle its motors without taking off. The results showed that there were frequency components appearing close to 448 kHz, coming from the UAV. No frequency components close to 457 kHz were recorded. This result is advantageous since the UAV wont pollute the spectrum at or close around 457 kHz. However, the UAV used in this measurement wont be the UAV used for the final project, thus more EMI measurements needs to be performed when the final UAV is available.

6.5 UART communication and Mavlink

The UART communication was tested using a logic analyzer. The data was display in an open source software called Sigrok PulseView. The Logic analyzer was connected to the UART TX port (#172) on the Odroid XU4. A snippet of the captured communication can be seen in figure 6.11 and this confirms that the communication works. Sigrok PulseView allows for decoding the sent bits, here ASCII decoding was used. For every successful detection of an avalanche beacon a so called "Mavlink message" is sent over UART. This Mavlink message has three data fields and a text field. The text field is set to send the characters 'Detection' to inform the UAV-system. The ASCII decoded bits of the text field saying 'Detection' can be seen in 6.11.



Figure 6.11: ASCII decoded bits from the UART communication using the Mavlink protocol.

7 Discussion

The radio receiver system is able to detect a transmitting avalanche beacons signal strength recorded from two antennas and forward this information to the UAV-system, thus the project goals has been met in those respects. The maximum range of detection was 15 meters inside an office where the radio environment was very noisy. In a less noisy environment, for instance the outdoors, where the system is supposed to be used, the detection range will be greater. If the power detection threshold could be lowered to say -125 dBm instead of -95 dBm, gaining 30 dB in dynamic range, one could expect an improvement of a factor $(1000)^{1/6} \approx 3.16$, increasing the maximum detection range close to 50 meters. These results are acceptable but certainly there is room for improvement.

One goal was not fully met in this master thesis with regards to calculate an angle towards the transmitting avalanche beacon. Tests were performed but the relative signal levels between the X and Y antenna detected from the beacon depended too heavily on the orientation of the avalanche beacon itself at the close distances where the measurements were performed. Perhaps the transmitting beacon signal will behave more isotropically at larger distances and not depend on its own orientation. Unfortunately time ran out and no measurements were performed at larger distances with regards to the angle estimation.

Regarding limitations of the developed system, only one active transmitting avalanche beacon was considered. It is certainly a possibility to have detections for more than one active beacon, but due to time constraints this was not implemented. According to the project goals the possibility for EMI from the UAV motors should be considered during the development of the system. Due to time constraints, no aspects of these possibilities have been considered during the development for the DSP-chain. Although, when measuring the EMI from a UAV the noise from that particular UAV did not appear at the frequency of 457 kHz and would not have caused problems for the system.

8 Conclusions

The finished radio receiver system works as a proof of concept and fulfills the majority of the project goals and should be considered successful. The only goal that was not met was the angle determination towards a transmitting avalanche beacon. The performance of the system is acceptable. A lot of time was spent in getting the whole concept to work as one system with the hardware interacting with the software. If time could be spent only on developing a powerful DSP-chain for the detection the system performance would improve in terms of maximum range of detection.

The suggestions for future work on this radio receiver system are:

- Refine the DSP-chain further to increase the maximum detection range.
- Implement detection for more than one transmitting avalanche beacon.
- Implement a feature that automatically sets the power level threshold for a positive detection.
- Further tests and investigation about determining an angle towards a transmitting beacon.
- Perform EMI measurements with the actual UAV that is supposed to be used in the end product.
- Evaluate the radio receiver together with the whole system including the UAV and UX.

References

- [1] Mandal, Mrinal and Asif, Amir. *Continuos and Discrete Time Signals and Systems*. University Press, Cambridge. 2007, 1st ed.
- [2] Pozar, David.M. *Microwave Engineering*. John Wiley & Sons, In. 2012, 4th ed.
- [3] Goldsmith, Andrea. *Wireless Communications*. University Press, Cambridge. 2005.
- [4] Rohde, Ulrich L., Whitaker, Jerry C. and Zahnd, Hans. *Communications Receivers - Principles and Designs*. McGraw-Hill Education. 2017, 4th ed.
- [5] AirSpy HF+. <https://airspy.com/airspy-hf-plus/>. Accessed: 2018-09-18.
- [6] Odroid XU4 manual. <https://magazine.odroid.com/odroid-xu4>. Accessed: 2018-09-18.
- [7] UART. <http://www.circuitbasics.com/basics-uart-communication/>. Accessed: 2019-01-07.
- [8] Sophocles J. Orfanidis. *Introduction to Signal Processing*. Sophocles J. Orfanidis. 2010. <https://www.ece.rutgers.edu/~orfanidi/intro2sp/>. Accessed: 2019-01-07.
- [9] Mavlink. <https://mavlink.io/en/>. Accessed: 2019-01-07.
- [10] IQ-modulation. https://www.keysight.com/upload/cmc_upload/All/IQ_Modulation.htm?cmpid=zzfindnw_iqmod&cc=GB&lc=eng. Accessed: 2019-01-07.
- [11] ZeroMQ. <http://zguide.zeromq.org/>. Accessed: 2019-01-07.
- [12] GNU Radio. <https://www.gnuradio.org/>. Accessed: 2018-09-03.
- [13] grOsmoSDR. <https://osmocom.org/projects/gr-osmosdr/wiki>. Accessed: 2018-09-03.
- [14] ETSI EN 300 718-1. https://www.etsi.org/deliver/etsi_en/300700_300799/30071801/02.01.00_20/en_30071801v020100a.pdf. Accessed: 2018-09-03.
- [15] Jackson, David, John. *Classical Electrodynamics*. Wiley: New York, 1998, 3rd ed.

Appendices

Matched filter software: detection_matched.py

```
1  #
2  #   Created by Richard Hedlund, richard.hedlund@protonmail.com
3  #   Master thesis project in Engineering Physics
4  #   Uppsala University
5  #
6
7
8  # Importing necessary libraries
9  import time
10 import zmq
11 import struct
12 import numpy as np
13 from pymavlink import mavutil
14
15 # Initialization of the ZeroMQ connection with GNU Radio
16 context = zmq.Context()
17
18 socketX = context.socket(zmq.PULL)
19 socketY = context.socket(zmq.PULL)
20
21 socketX.connect("tcp://127.0.0.1:5555")
22 socketY.connect("tcp://127.0.0.1:5556")
23
24 # Connection to the UART/Serial bus of the Odroid XU4
25 the_connection = mavutil.mavlink_connection('/dev/ttySAC0', 115200)
26
27 # Power threshold which is tested and measured using a lab bench test.
28 # power_threshold_dBm will need to be adjusted according to the noise
29 #   ↪ environment of the radio receiver.
30 power_threshold_dBm = -135
31
32 # Calibration factor due to the fact that the SDRs measure slightly
33 #   ↪ different amplitude values
34 calib_factor = 1.044
35
36 while True:
37
38     messageX = socketX.recv()
39     messageY = socketY.recv()
40
41     dataX = np.fromstring(messageX, dtype = np.float32)
42     dataY = np.fromstring(messageY, dtype = np.float32)
```

```
42     max_X = max(dataX)
43     max_Y = max(dataY)
44
45     dataXX = np.square(dataX)
46     dataYY = np.square(dataY)
47
48     max_X = max(dataXX)
49     max_Y = max(dataYY)
50
51     X_dBm = 10*np.log10(max_X) - 30
52     Y_dBm = 10*np.log10(max_Y) - 30
53
54     if X_dBm > power_threshold_dBm or Y_dBm > power_threshold_dBm:
55
56         # Time of detection
57         time_usec = time.time()
58         print("Max X: " + str(X_dBm) + " dBm")
59         print("Max Y: " + str(Y_dBm) + " dBm")
60
61         # Calculate the angle between the X and Y signal strength
62         angle = np.arctan2(max_Y,max_X)*360/(2*np.pi)
63
64         # Mavlink protocol output over UART
65         the_connection.mav.debug_vect_send('Detection', time_usec, X_dBm,
66         ↪ Y_dBm, angle)
67
68
69
```

FFT software: detection_fft.py

```
1  #
2  #   Created by Richard Hedlund, richard.hedlund@protonmail.com
3  #   Master thesis project in Engineering Physics
4  #   Uppsala University
5  #
6
7
8  # Importing necessary libraries
9  import time
10 import zmq
11 import struct
12 import numpy as np
13 from pymavlink import mavutil
14
15 # Initialization of the ZeroMQ connection with GNU Radio
16 context = zmq.Context()
17
18 socket1 = context.socket(zmq.PULL)
19 socket2 = context.socket(zmq.PULL)
20
21 socket1.connect("tcp://127.0.0.1:5555")
22 socket2.connect("tcp://127.0.0.1:5556")
23
24 # Initialization of the UART connection
25 the_connection = mavutil.mavlink_connection('/dev/ttySAC0', 115200)
26
27 # Power threshold which is tested and measured using a lab bench test.
28 # power_threshold_dBm will need to be adjusted according to the noise
29 #   ↪ environment of the radio receiver.
30 power_threshold_dBm = -115
31
32 # Calibration factor due to the fact that the SDRs measure slightly
33 #   ↪ different amplitude values
34 calib_factor = 1.044
35
36 # Buffers for data
37 buffer1 = np.zeros(4, dtype = np.float32)
38 buffer2 = np.zeros(4, dtype = np.float32)
39 buff_ctr = 0
40
41 buffer_flag = False
42
43 while True:
```

```

43      # The program will wait here until it receives data from GNU Radio,
      ↪ only when data is received the program will continue
      ↪ automatically.
44      message1 = socket1.recv()
45      message2 = socket2.recv()
46
47      # Converting the binary data from GNU Radio to 32 bit float value
      ↪ arrays.
48      arr1 = np.fromstring(message1, dtype = np.float32)
49      arr2 = np.fromstring(message2, dtype = np.float32)*calib_factor
50
51      # Only looking in the necessary frequency bins (see report for more
      ↪ info).
52      arr1_max = max(arr1[1:7]) # 457 kHz +- 90 Hz
53      arr2_max = max(arr2[1:7]) # 457 kHz +- 90 Hz
54
55      # Converting to dBm. (Calibrated with signal generator).
56      arr1_max_dBm = 10*np.log10(arr1_max) - 30
57      arr2_max_dBm = 10*np.log10(arr2_max) - 30
58
59      # If signal detected, buffer the 3 next incoming measurements.
      ↪ Because of the pulse width assumed to be approx 100ms.
60      # Every frequency bin in the FFT is 30 Hz -> 33.333... ms
61      if arr1_max_dBm > power_threshold_dBm and arr2_max_dBm and buff_ctr
      ↪ <=3 and buffer_flag == False:
62
63          buffer_flag = True
64
65      if buffer_flag == True:
66
67          if buff_ctr < 4:
68              buffer1[buff_ctr] = arr1_max
69              buffer2[buff_ctr] = arr2_max
70              buff_ctr = buff_ctr + 1
71
72          else:
73              time_usec = time.time()
74
75              x_sig_str = max(buffer1)
76              y_sig_str = max(buffer2)
77
78              x_sig_str_dBm = 10*np.log10(x_sig_str) -30
79              y_sig_str_dBm = 10*np.log10(y_sig_str) -30
80
81              angle = np.arctan2(y_sig_str,x_sig_str)*360/(2*np.pi)
82
83      # Mavlink protocol output over UART

```

```
84         the_connection.mav.debug_vect_send('Detection', time_usec,  
      ↪     x_sig_str_dBm, y_sig_str_dBm, angle)  
85  
86         buff_ctr = 0  
87         buffer_flag = False
```

Filter bank software: detection_fbanks.py

```
1  #
2  #   Created by Richard Hedlund, richard.hedlund@protonmail.com
3  #   Master thesis project in Engineering Physics
4  #   Uppsala University
5  #
6
7
8  # Importing necessary libraries
9  import time
10 import zmq
11 import struct
12 import numpy as np
13 from pymavlink import mavutil
14
15 from Antenna import Antenna
16
17 # Initialization of the ZeroMQ connection with GNU Radio
18 context = zmq.Context()
19
20 socketX = context.socket(zmq.PULL)
21 socketY = context.socket(zmq.PULL)
22
23 socketX.connect("tcp://127.0.0.1:5555")
24 socketY.connect("tcp://127.0.0.1:5556")
25
26 # Initialization of the UART connection
27 the_connection = mavutil.mavlink_connection('/dev/ttySAC0', 115200)
28
29 vect_size = 1
30
31 antennaX = Antenna(np.zeros(11*vect_size, dtype =
    ↪ np.float32), 0.0, 0.0, 0.0, 0.0, 0.0, np.zeros(3, dtype = np.float32),
    ↪ [0]*3, 0, 0.0, 0.0, 0)
32 antennaY = Antenna(np.zeros(11*vect_size, dtype =
    ↪ np.float32), 0.0, 0.0, 0.0, 0.0, 0.0, np.zeros(3, dtype = np.float32),
    ↪ [0]*3, 0, 0.0, 0.0, 0)
33
34 antennaX_flag = False
35 antennaY_flag = False
36
37 calib_factor = 1.044
38
39 x_prev = 0.0
40 y_prev = 0.0
41
42 sign_threshold = 0.01
```

```
43
44 while True:
45
46     messageX = socketX.recv()
47     messageY = socketY.recv()
48
49     dataX = np.fromstring(messageX, dtype = np.float32)
50     dataY = np.fromstring(messageY, dtype = np.float32)
51
52     antennaX.data = np.array(dataX)
53     antennaY.data = np.array(dataY)*calib_factor
54
55
56     if antennaX_flag == False:
57
58         if antennaX.detection() == True:
59             antennaX_flag = True
60             x_sig_str = antennaX.def_max
61             x_sig_str_dB = antennaX.def_max_dB
62             x_sig_filter = antennaX.def_filter
63             x_sig_meas_ctr = antennaX.meas_ctr
64
65     if antennaY_flag == False:
66         if antennaY.detection() == True:
67             antennaY_flag = True
68             y_sig_str = antennaY.def_max
69             y_sig_str_dB = antennaY.def_max_dB
70             y_sig_filter = antennaY.def_filter
71             y_sig_meas_ctr = antennaY.meas_ctr
72
73     if antennaX_flag == True and antennaY_flag == True:
74
75         angle = np.arctan2(y_sig_str,x_sig_str)*360/(2*np.pi)
76         time_usec = time.time()
77
78         # Send this data over UART using debug_vect
79         the_connection.mav.debug_vect_send('Detection', time_usec, x_dir,
80             ↪ y_dir, angle)
81
82         # Set detection flags to False
83         antennaX.meas_ctr = 0
84         antennaX.prev_max_mean = 0.0
85         antennaX_flag = False
86
87         antennaY.meas_ctr = 0
88         antennaY.prev_max_mean = 0.0
89         antennaY_flag = False
```

Antenna.py (class used in detection_fbanks.py)

```

1  import numpy as np
2
3  class Antenna:
4
5      power_threshold_dB = -120
6      vect_size = 1
7
8      # Define what an Antenna is in the initialization function
9      def __init__(self,
10                  data,
11                  curr_max_mean,
12                  curr_max_mean_dB,
13                  curr_max_index,
14                  prev_max_mean,
15                  prev_max_index,
16                  meas_ctr,
17                  max_str_buffer,
18                  max_ind_buffer,
19                  max_ctr,
20                  def_max,
21                  def_max_dB,
22                  def_filter
23              ):
24
25          self.data = data
26          self.curr_max_mean = curr_max_mean
27          self.curr_max_mean_dB = curr_max_mean_dB
28          self.curr_max_index = curr_max_index
29
30          self.prev_max_mean = prev_max_mean
31          self.prev_max_index = prev_max_index
32
33          self.meas_ctr = meas_ctr
34          self.max_str_buffer = max_str_buffer
35          self.max_ind_buffer = max_ind_buffer
36          self.max_ctr = max_ctr
37
38          self.def_max = def_max
39          self.def_max_dB = def_max_dB
40          self.def_filter = def_filter
41
42      def detection(self):
43
44          power_threshold_dB = -120
45
46          filter_bank_mean = np.zeros(11, dtype = np.float32)

```

```

47
48     filter_bank_mean[0] = self.data[0]
49     filter_bank_mean[1] = self.data[1]
50     filter_bank_mean[2] = self.data[2]
51     filter_bank_mean[3] = self.data[3]
52     filter_bank_mean[4] = self.data[4]
53     filter_bank_mean[5] = self.data[5]
54     filter_bank_mean[6] = self.data[6]
55     filter_bank_mean[7] = self.data[7]
56     filter_bank_mean[8] = self.data[8]
57     filter_bank_mean[9] = self.data[9]
58     filter_bank_mean[10] = self.data[10]
59
60
61     self.curr_max_mean      = max(filter_bank_mean)
62     self.curr_max_index     = np.argmax(filter_bank_mean)
63     self.curr_max_mean_dB = 10*np.log10(self.curr_max_mean)-30
64
65     if self.curr_max_mean > self.prev_max_mean and
66     ↪ self.curr_max_mean_dB > power_threshold_dB:
67         self.prev_max_mean = self.curr_max_mean
68         self.prev_max_index = self.curr_max_index
69         self.prev_max_mean_dB = self.curr_max_mean_dB
70         self.meas_ctr = self.meas_ctr + 1
71
72         return False
73
74     if self.curr_max_mean < self.prev_max_mean and
75     ↪ self.prev_max_mean_dB > power_threshold_dB:
76
77         if self.max_ctr <= 2:
78             self.max_str_buffer[self.max_ctr] = self.prev_max_mean
79             self.max_ind_buffer[self.max_ctr] = self.prev_max_index
80             self.max_ctr = self.max_ctr + 1
81
82             return False
83         else:
84             self.max_ctr = 0
85             self.def_max = max(self.max_str_buffer)
86             self.def_max_dB = 10*np.log10(self.def_max)-30
87             self.def_filter = max(self.max_ind_buffer)
88             return True

```